SafeBite: An AI-Driven Mobile Application for Personalized Food Ingredient Analysis, Health and Nutrition Recommendations Using RAG, LLMs and Context Intelligence

FYP Code: 23F-28

Software Design Specification

BY

AMJAD ALI ABRO | CMS ID:023-23-0043
SARFRAZ ALI KATPAR | CMS ID: 023-23-0045
SEHRISH KHEMANI | CMS ID: 023-23-0224



Supervised by: Dr. Saif Hassan _____

Co-Supervised by: Prof. Dr. Sher Muhammad Daudpota _____

DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF SCIENCE AND INFORMATION TECHNOLOGY
SUKKUR IBA UNIVERSITY

NOVEMBER 2026

Table of Contents

1. Introduction 2
1.1. Purpose of Document 3
1.2. Definitions, acronyms, and abbreviations 3
2. Overall System Description 4
2.1. Project Background 4
2.2. Project Objectives 4
3.1. Architectural Overview 5
3.2. Component Descriptions 8
4. Domain Model 10
5. Class Diagram 11
6. Database Diagram 14
7. Entity Relationship Diagram (ERD) 17
8. Sequence Diagram(s) 19
Sign-in and Session Start 20
UC-01 / UC-06 : Create or Update Food Profile and Manage Personal Avoid List 22
UC-02: Analyze a Packaged Food Product 23
UC-03: Explore an Ingredient 27
UC-04: Compare Products 29
UC-05: Scan and Review a Health Report 31
UC-07: Review or Delete Scan History 34
Knowledge and Rule Maintenance (SRS Module 9) 36
9. System Interface Design 38
10. Test Cases 42
10.1. Functional Test Cases 43
10.2. Non-Functional Test Cases 46

1. Introduction

1.1. Purpose of Document

This Software Design Specification (SDS) describes how SafeBite will be designed so that the requirements in the Software Requirements Specification (SRS) and the scope in the approved Proposal can be implemented and tested. The SRS explains what the system must do. This document explains how the parts of the system will be organized, what each part is responsible for, how data will be stored, and how the parts will interact during the main workflows.

The document covers the system architecture, domain model, class diagram, database design, entity relationship diagram, sequence diagrams, interface design, and the list of test cases. The intended readers are the supervisor, the co-supervisor, the departmental evaluators, the three team members, and anyone who may maintain the system after the FYP.

1.2. Definitions, acronyms, and abbreviations

Table 1: Definitions, acronyms, and abbreviations

Term	Description
AI	Artificial Intelligence
API	Application Programming Interface
OCR	Optical Character Recognition; extraction of text from an image
LLM	Large Language Model
RAG	Retrieval-Augmented Generation; an LLM answer produced from retrieved reference text
PHSS	Personal Health Suitability Score. A rule-based, project-defined score; PHSS = 0.40H + 0.30N + 0.20D + 0.10G. It is a decision-support indicator and not a clinical measure
H, N, D, G	PHSS factors: Health-Related Dietary Needs Compatibility, Nutrition Goals Compatibility, Dietary Preference Compatibility, General Ingredient Suitability
Safe*	PHSS category for scores of 80 to 100. The asterisk means that no conflict was found in the available information; it is not a guarantee of safety
Caution / Avoid	PHSS categories for scores of 50 to 79 and 0 to 49
Unable to verify	A separate result state shown when critical information is missing, unreadable, or contradictory. It is not one of the three PHSS categories
Hard constraint	A rule that overrides the numerical score, for example a declared allergy found in the ingredients
Personal avoid list	Ingredients, additives or terms that the user chooses to avoid, kept separate from declared allergies
Hybrid barcode/OCR	Product identification that uses a barcode database record and OCR of the physical label together, and flags differences between them

Term	Description
Ingredient Explorer	The searchable view of an ingredient with its aliases, explanation and sources
Knowledge base (KB)	The structured ingredient and nutrition reference data used for matching and retrieval
FAISS	Facebook AI Similarity Search; a library for vector similarity search
JSON / REST / ORM	JavaScript Object Notation / Representational State Transfer / Object-Relational Mapping
HTTPS	Hypertext Transfer Protocol Secure
FR / NFR / UC	Functional Requirement / Non-Functional Requirement / Use Case, as numbered in the SRS
JWT / ID token	A signed token issued by the authentication service and verified by the backend

2. Overall System Description

2.1. Project Background

Food packages carry ingredient lists, nutrition panels and allergen statements, but many people find this information hard to read. Ingredient names can be technical or abbreviated, and whether an ingredient matters depends on the person. A product that is fine for one user may not be suitable for another because of an allergy, a dietary need, a nutrition goal or a personal preference. Barcode-based apps help with lookup, but a database record can be incomplete or may not match the package in the user's hand. Users may also keep laboratory reports, such as lipid profile results, separately from their food decisions.

SafeBite is a mobile application that brings these pieces into one workflow. It reads the physical package with OCR, retrieves a product record using the barcode, compares the two, matches ingredients to a structured knowledge base, and checks the result against the user's profile and personal avoid list. It also provides source-grounded ingredient explanations using RAG and an LLM, a rule-based PHSS, product comparison, an optional lipid profile report scanner, and a user-controlled scan history. The application supports informed choices. It does not diagnose any medical condition or replace the advice of a doctor, allergist or dietitian.

This SDS keeps the safety position of the Proposal and SRS: an allergy conflict is never hidden by a good score, an unclear result is shown as Unable to verify instead of Safe, and report values are only saved when the user chooses to save them.

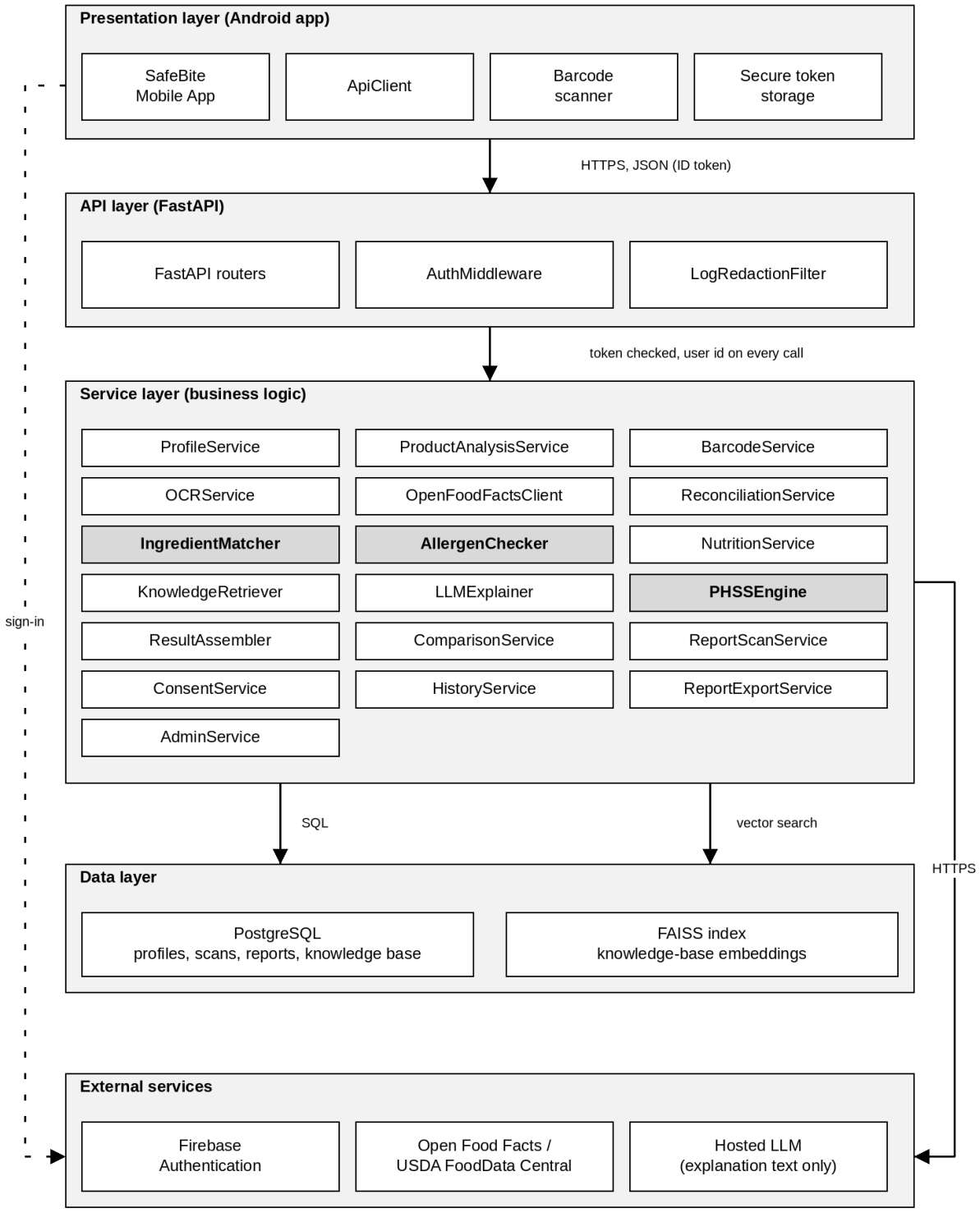
2.2. Project Objectives

- **Product review workflow:** Provide a mobile workflow for reading packaged-food ingredients and the available nutrition information (FR-02, FR-03).
- **Hybrid verification:** Combine barcode-based product retrieval with OCR of the actual label and make any mismatch visible to the user (FR-02, FR-03).

- **Personal constraints:** Identify matches against declared allergies and the personal avoid list and show them as prominent warnings (FR-01, FR-04).
- **Grounded explanations:** Give ingredient explanations that come from retrieved reference material, using RAG and an LLM only for the wording (FR-06).
- **Documented scoring:** Calculate the PHSS with documented and testable rules, and show its limits (FR-05).
- **Comparison:** Compare two or more products against the user's profile using only the data that is available (FR-07).
- **Report scanning:** Extract lipid profile values from a report as an optional feature, only after separate consent and user review (FR-08, FR-10).
- **Privacy and control:** Protect profile and report data through authentication, per-user access control, consent, and a history that the user can review and delete (FR-09, NFR-SEC group).
- **Evaluation:** Make the system testable for extraction quality, rule correctness, evidence grounding, safe handling of uncertainty and usability (Section 10).

3.1. Architectural Overview

SafeBite follows a client-server design with a layered backend. The mobile application handles capture and display. The backend, written in FastAPI, performs all processing: OCR, reconciliation, ingredient matching, retrieval, scoring and storage. The application does not run safety decisions on the phone, so the rules stay in one place and give the same result for the same input (NFR-PER-04). The following figure shows the five layers and the data flow between them.



Deterministic rule components (matching, hard constraints, scoring). The LLM never decides these.

The app talks only to the API layer. Only the service layer reaches the database, the index and the LLM.

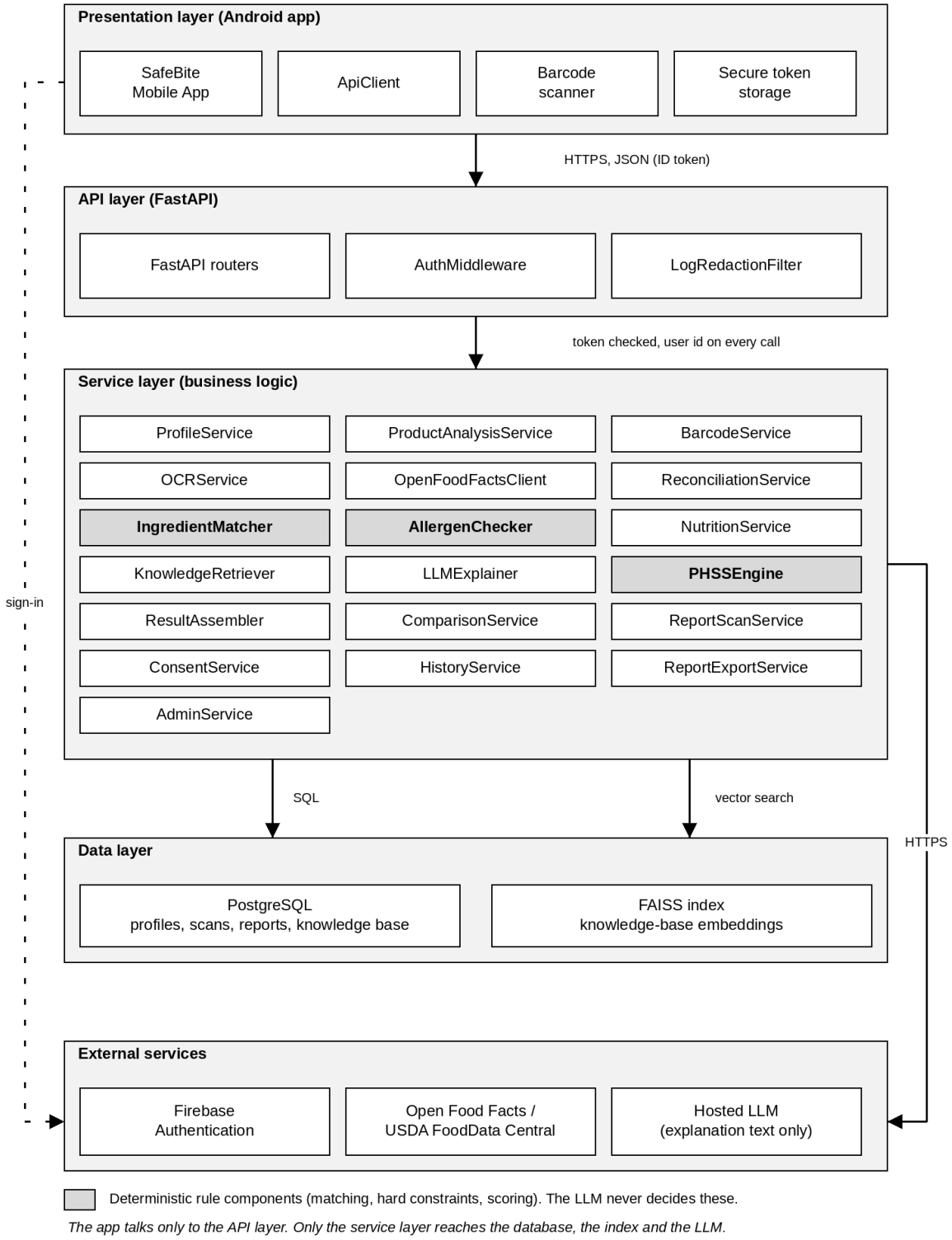


Figure 1: High-level architecture of SafeBite showing the layers and data flow.

Description: The architecture diagram shows how the SafeBite components work together. The app talks only to the API layer over HTTPS. The API layer validates every request, verifies the Firebase ID token and calls the services. Matching, hard constraints and scoring (orange boxes) are deterministic services, and

the LLM is called only to write an explanation. Only the service layer reaches PostgreSQL, the FAISS index and the external services. Sign-in is the only call the phone makes directly to an external service (Firebase).

3.2. Component Descriptions

Table 2 lists the components that appear in the architecture and in the rest of the document. The same names are used in the class diagram and the sequence diagrams.

Table 2: System components and responsibilities

Component	Layer	Responsibility	Main requirements
SafeBite Mobile App	Presentation	Screens, camera capture, barcode scanning, consent prompts, result display, warnings, retry on timeout	FR-01 to FR-10, NFR-PER-02, NFR-PER-03
ApiClient	Presentation	Sends HTTPS requests with the ID token, handles timeouts and errors	NFR-PER-03, NFR-SEC-03
AuthMiddleware	API	Verifies the Firebase ID token and sets the current user for each request	FR-01, NFR-SEC-01, NFR-SEC-02
ProfileService	Service	Stores and validates the food profile and personal avoid list	FR-01, UC-01, UC-06
ProductAnalysisService	Service	Coordinates the product analysis workflow from input to result	FR-02 to FR-05, UC-02
BarcodeService	Service	Decodes barcodes from uploaded images when the app does not send a decoded value	FR-02
OCRService	Service	Preprocesses images and extracts label text with confidence values	FR-02, FR-03
OpenFoodFactsClient	Service	Retrieves the product record for a barcode; marks missing fields as unavailable	FR-02, FR-03
ReconciliationService	Service	Normalizes database and OCR fields, and flags conflicts and low confidence	FR-03
IngredientMatcher	Service	Maps detected ingredient text to canonical ingredients: exact, alias, fuzzy, then semantic matching	FR-04, NFR-PER-04
AllergenChecker	Service	Compares matched ingredients against declared allergies and the avoid list	FR-04, NFR-SAF-02
NutritionService	Service	Gathers available nutrition fields; optionally uses USDA FoodData Central	FR-05

Component	Layer	Responsibility	Main requirements
KnowledgeRetriever	Service	Embeds queries and retrieves evidence from the FAISS index and structured KB tables	FR-04, FR-06
LLMExplainer	Service	Writes plain-language explanations only from retrieved text; never decides the score or an allergen result	FR-06, NFR-SAF-04
PHSSEngine	Service	Applies the rule tables, computes H, N, D, G and the PHSS, and applies hard constraints	FR-05, NFR-PER-04, NFR-SAF-02, NFR-SAF-08
ResultAssembler	Service	Builds the result with warnings, sources, uncertainty indicators and the result state	FR-05, NFR-SAF-03, NFR-SAF-04
ComparisonService	Service	Aligns fields of two or more products, applies hard constraints, then ranks	FR-07, UC-04
ReportScanService	Service	Extracts lipid profile candidates, requires review, and saves only on confirmation	FR-08, FR-10, NFR-SAF-05, NFR-SAF-07
ConsentService	Service	Records report consent decisions and blocks report processing without consent	FR-08, FR-10, NFR-SEC-04, NFR-SEC-05
HistoryService	Service	Lists, opens and deletes saved scans and report values; produces optional summaries	FR-09, NFR-SEC-06
ReportExportService (optional)	Service	Creates a downloadable PDF summary with ReportLab	Proposal Table 8
AdminService	Service	Lets an administrator maintain ingredient records, rule tables and data-source settings	SRS 4.1, module 9
LogRedactionFilter	Cross-cutting	Removes passwords, tokens and report contents from logs	NFR-SEC-07
PostgreSQL	Data	Stores users, profiles, consent, scans, reports and knowledge-base records	All stored data
FAISS index	Data	Vector index of knowledge-base chunks built with all-MiniLM-L6-v2 embeddings	FR-06
Firebase Authentication	External	Sign-up, sign-in and token issuing; the SafeBite backend never receives passwords	FR-01, NFR-SEC-03

Component	Layer	Responsibility	Main requirements
Open Food Facts / USDA FoodData Central	External	Product and nutrition reference data	FR-02, FR-05
Hosted LLM (Gemini, GPT-4o-mini or similar)	External	Text generation behind the LLMExplainer interface	FR-06

(Assumption: Firebase Authentication is used, as listed in the Proposal. The SRS only requires an established authentication provider, so the AuthMiddleware is written so that another provider can replace it.)

(Assumption: the Administrator role is a user account with an administrator flag. The SRS lists Administration and Knowledge Maintenance as a module but gives no functional requirement for it, so the admin screens are kept small and are limited to ingredient records, rule tables and data-source settings.)

4. Domain Model

The domain model gives a clear picture of the SafeBite system. It shows the main concepts and how they relate, without implementation detail. It is drawn from the data described in the Proposal and the SRS. The same names are used in the class diagrams (Section 5), the database diagrams (Section 6) and the ERDs (Section 7).

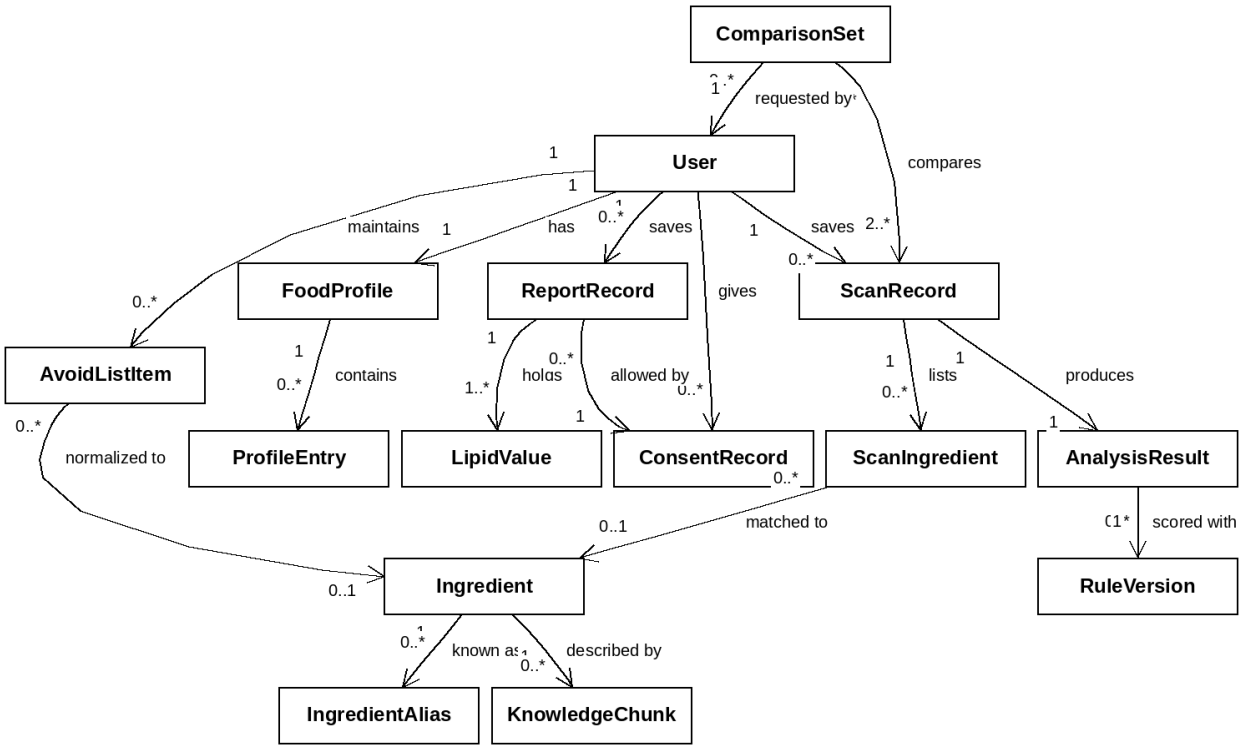


Figure 2: Conceptual domain model of SafeBite.

Description: The domain model shows how users, profiles, scans, results, reports and the ingredient knowledge base are connected, with the multiplicity of each relationship. The model keeps two kinds of user entries separate: declared allergies are part of the FoodProfile, while the AvoidListItem holds ingredient terms. Product records fetched from Open Food Facts are not a separate stored concept; the fields used are kept inside the ScanRecord so that a past scan can be reopened as it was analysed. A ComparisonSet is temporary and is not stored.

5. Class Diagram

The class diagram displays the object-oriented design of SafeBite. It shows the entity classes that hold data and map to the database tables in Section 6, with their attributes, operations and associations. To keep the text readable, the diagram is shown in three parts: user data (Figure 3), scans and results (Figure 4), and the ingredient knowledge base (Figure 5). Dashed boxes show classes that are defined in another part. The service classes that carry out the workflows are the components listed in Table 2. Class and attribute names follow the domain model in Section 4. Types are written in a general form; the Python implementation will use SQLModel or SQLAlchemy models and Pydantic schemas for the same structures (Assumption).

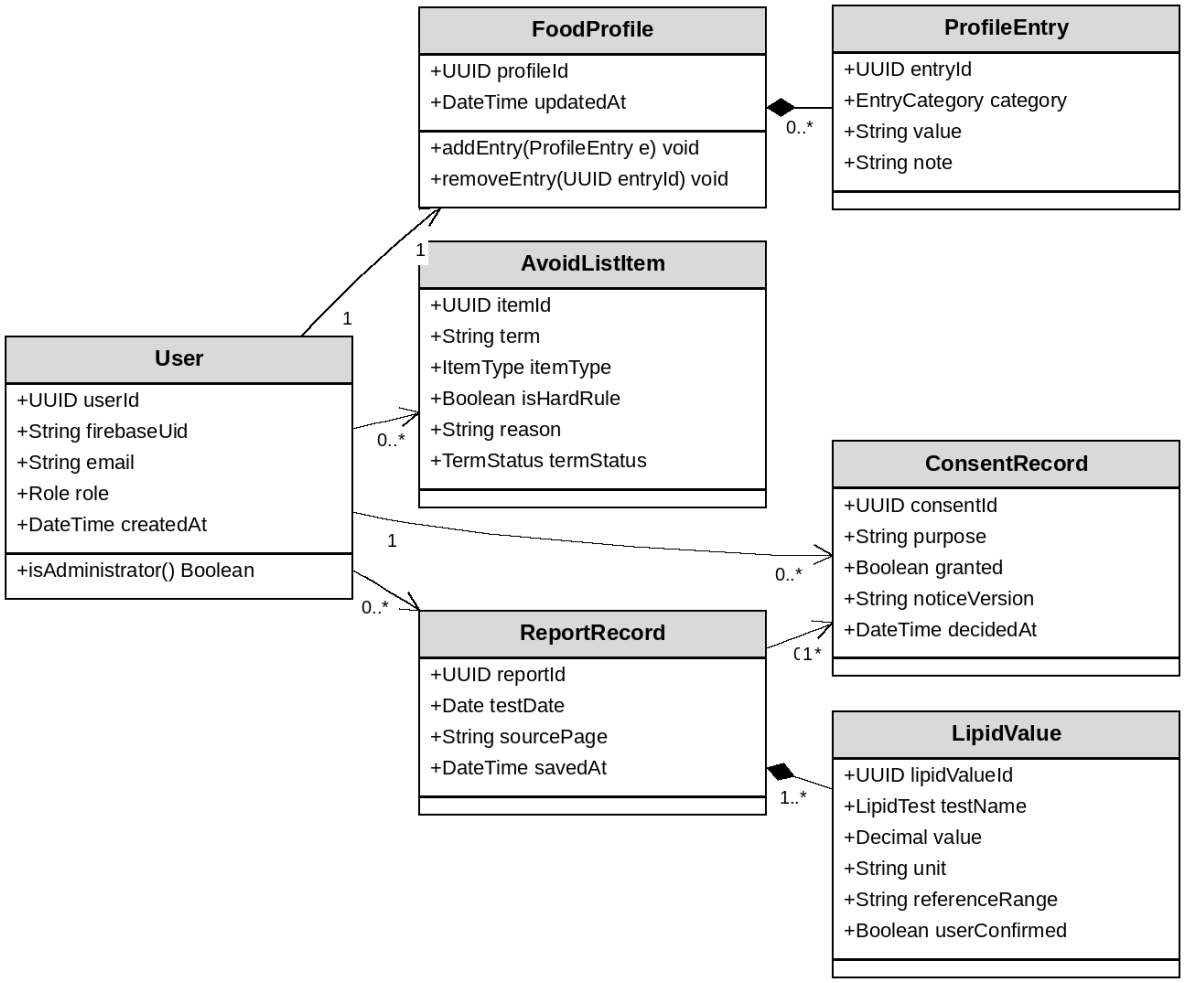


Figure 3: SafeBite UML class diagram: user data.

Description: Figure 3 shows the User with its FoodProfile, personal AvoidListItems, ConsentRecords and ReportRecords. The filled diamond means composition: a FoodProfile owns its ProfileEntries and a ReportRecord owns its LipidValues. A ReportRecord always refers to the ConsentRecord that allowed it.

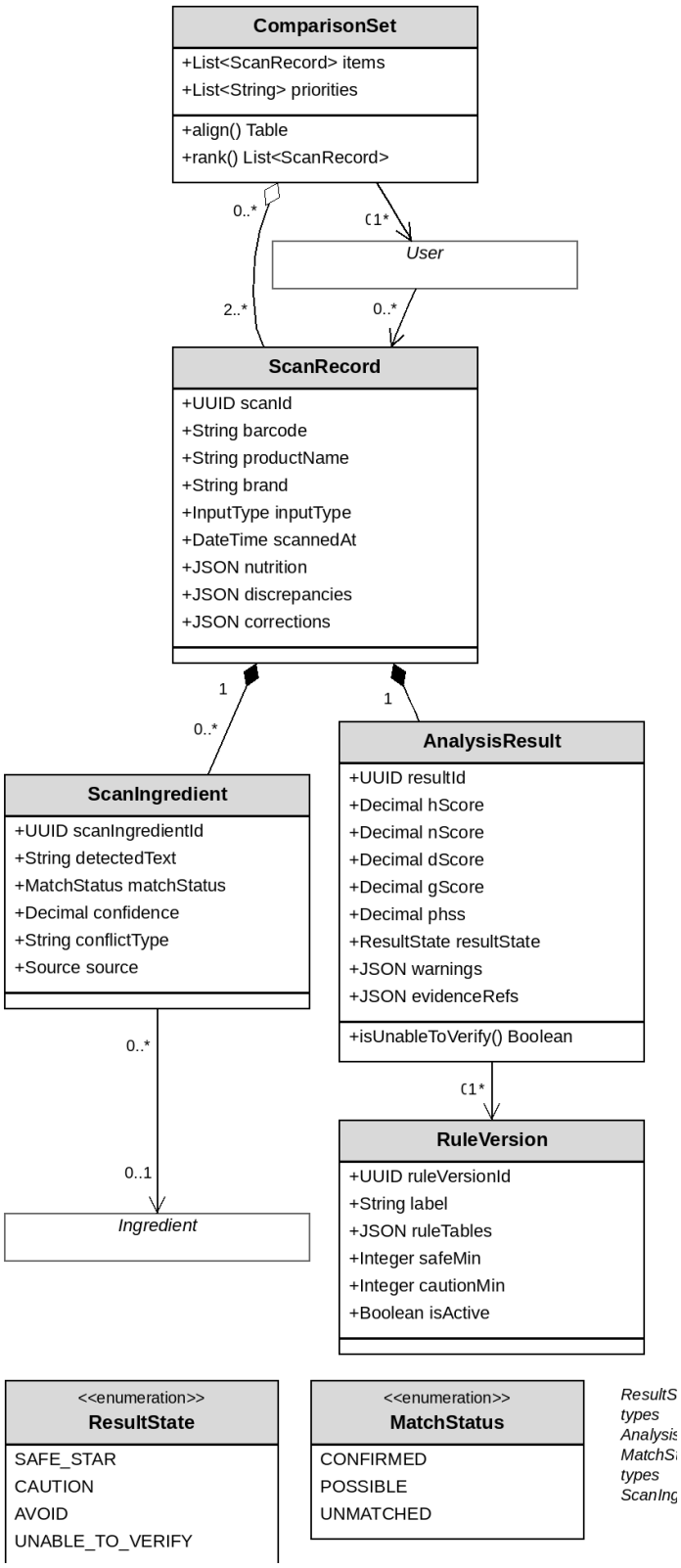


Figure 4: SafeBite UML class diagram: scans and results.

Description: Figure 4 shows the ScanRecord, which owns its ScanIngredients and its single AnalysisResult. An AnalysisResult records the RuleVersion it was scored with. The hollow diamond shows that a temporary ComparisonSet refers to two or more ScanRecords. The enumerations ResultState and MatchStatus fix the allowed values of the result state and the match status.

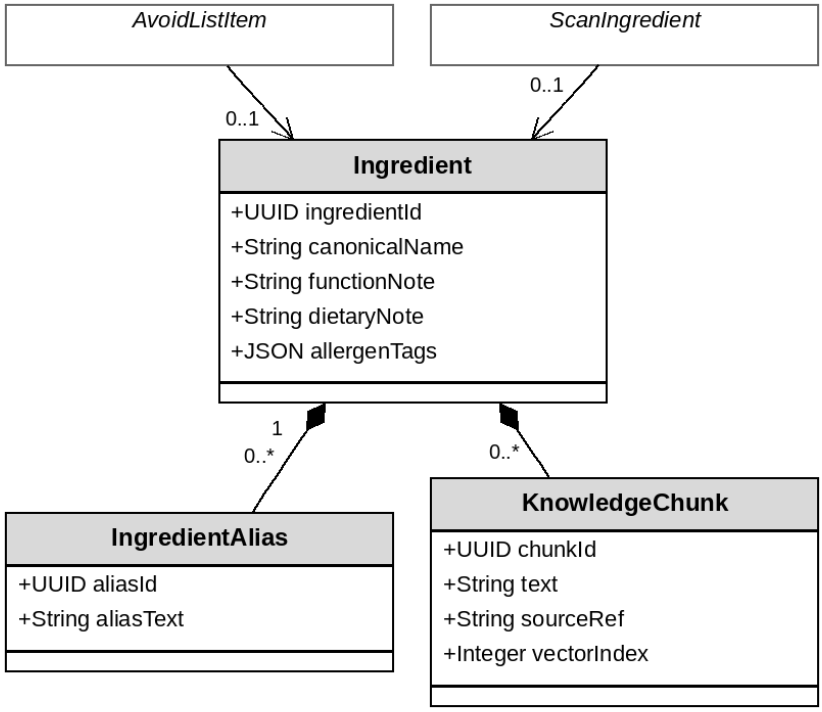


Figure 5: SafeBite UML class diagram: ingredient knowledge base.

Description: Figure 5 shows the Ingredient with its aliases and knowledge chunks. An Ingredient may be linked from an AvoidListItem or a ScanIngredient; the link is optional because a user-defined or unmatched term has no canonical record.

6. Database Diagram

SafeBite stores its data in PostgreSQL, accessed from the FastAPI backend through SQLAlchemy or SQLModel. Figures 6 to 8 show the physical database design: tables, columns, data types, keys and relationships. Table names are the plural snake_case form of the entity classes in Section 5. The 15 tables are shown in three parts, and tables that belong to another part appear as dashed boxes.

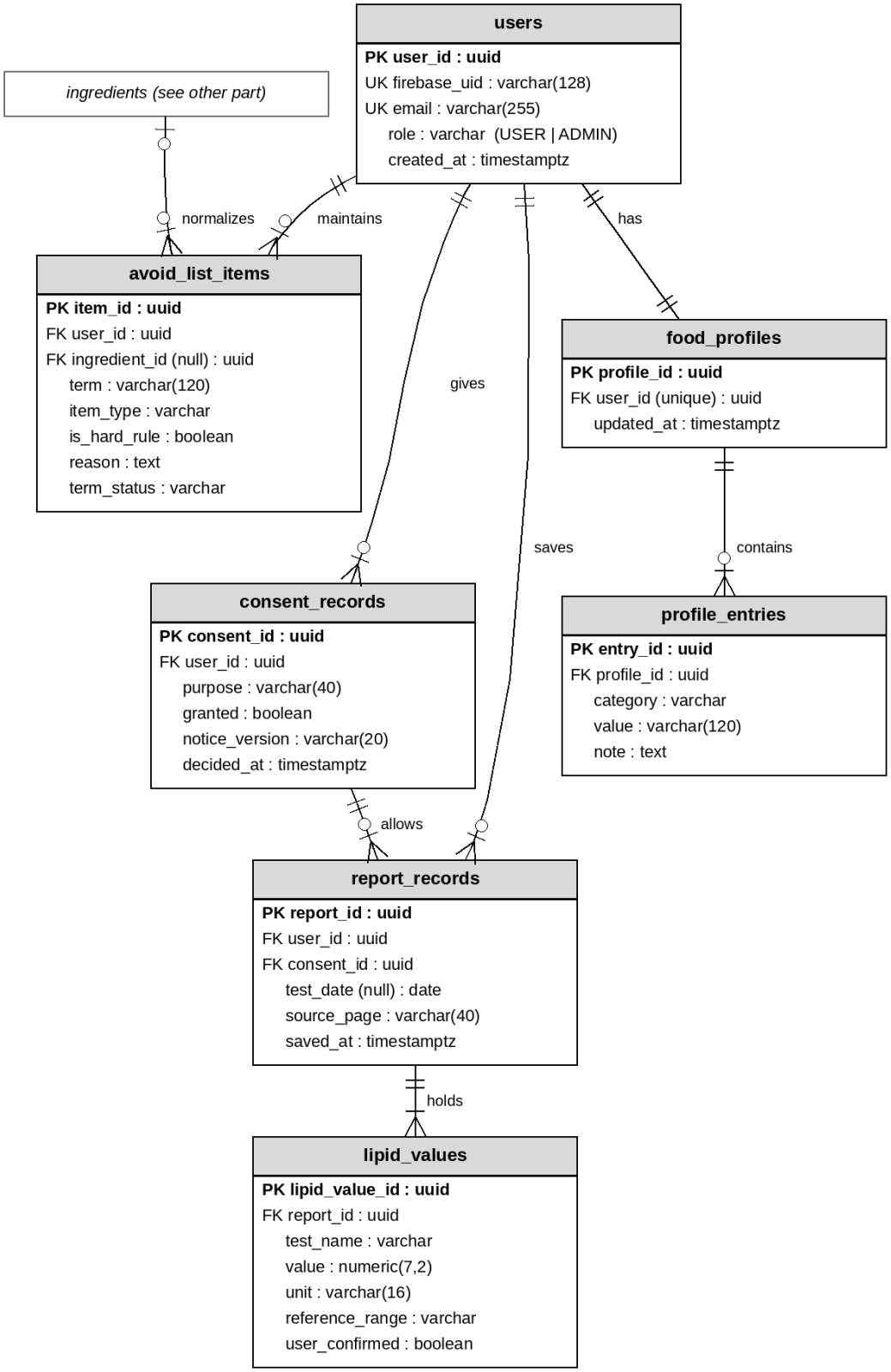


Figure 6: SafeBite database diagram: users, profile and reports.

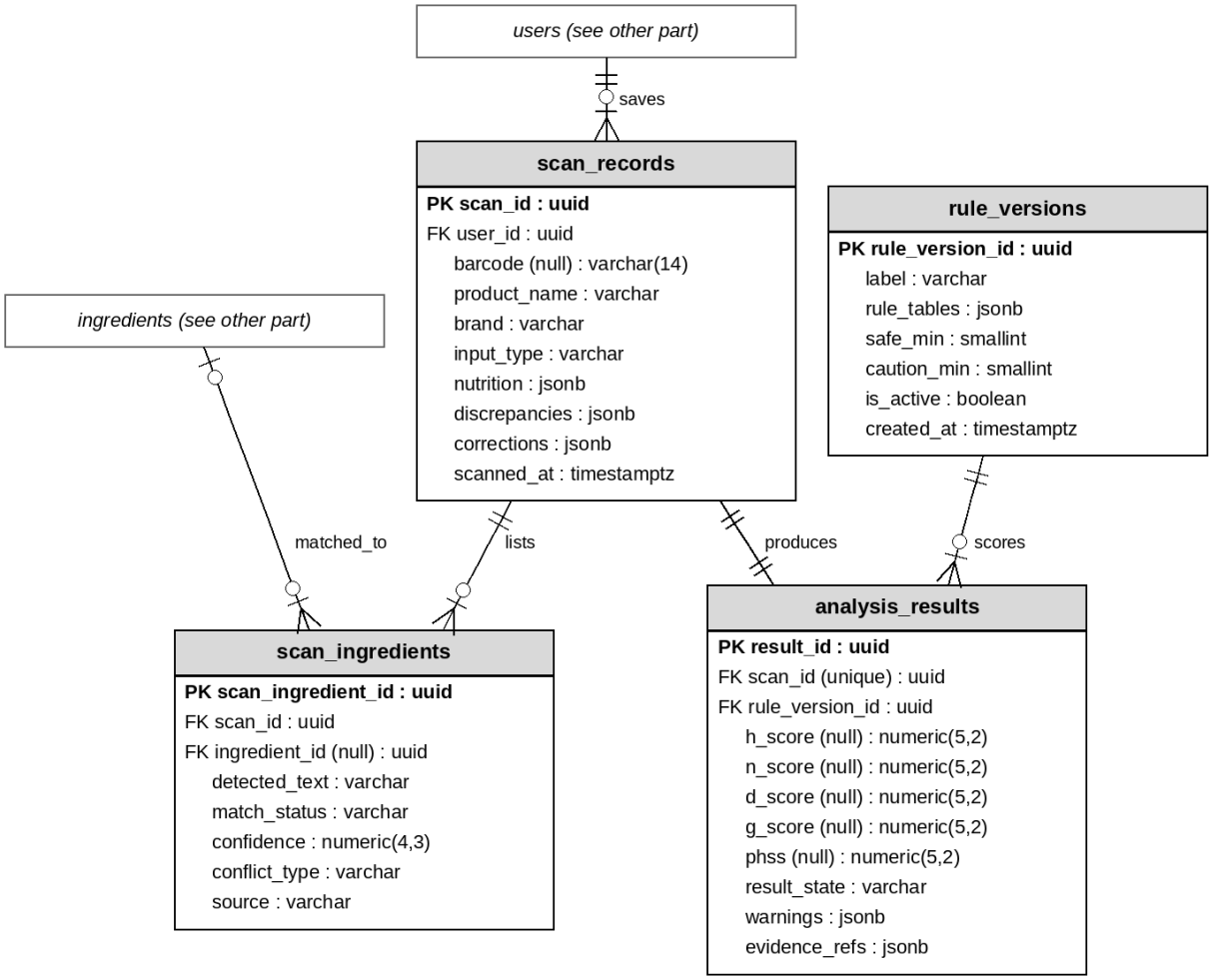


Figure 7: SafeBite database diagram: scans and results.

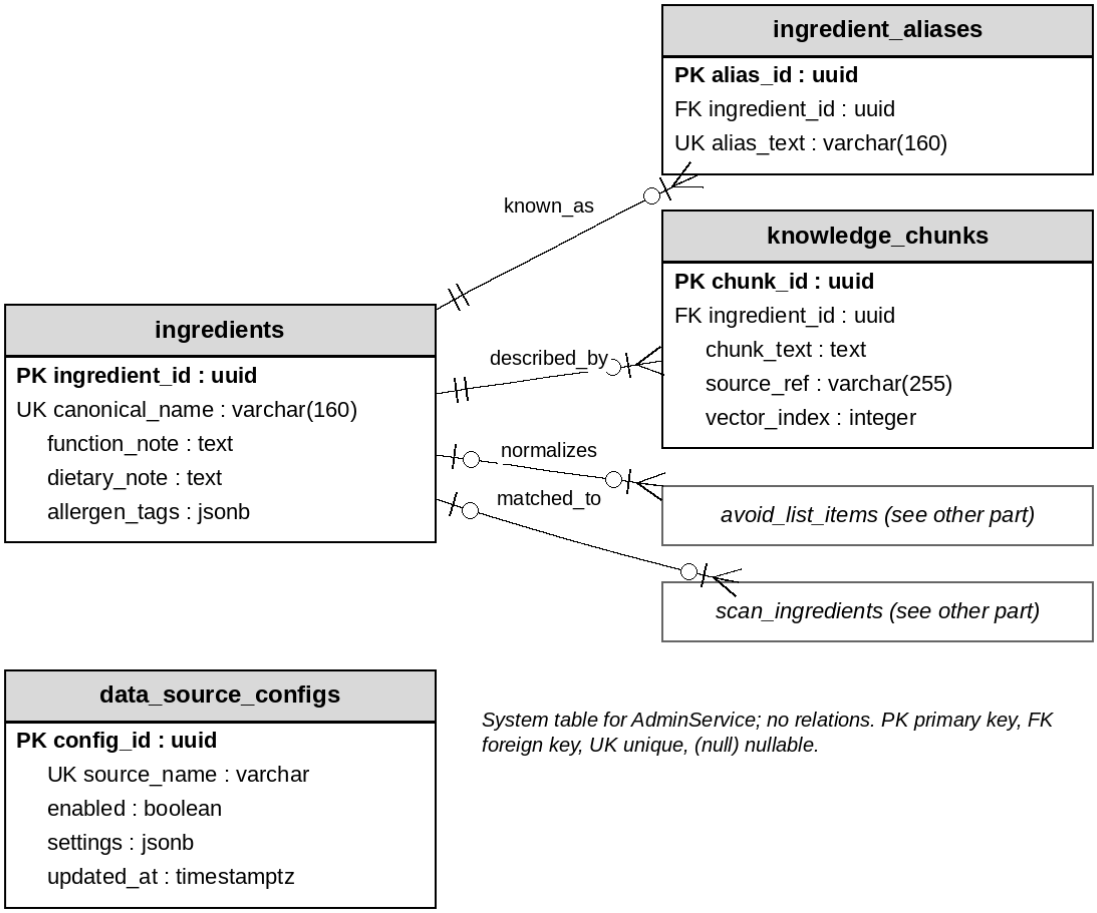


Figure 8: SafeBite database diagram: ingredient knowledge base.

Description: The database diagrams show the primary keys (PK), foreign keys (FK) and unique columns (UK) of each table, and the relationship lines use crow's foot notation. The table data_source_configs is a system table for the AdminService; it keeps non-secret settings such as which external source is enabled. API keys are not stored in it; they stay in server-side configuration (NFR-SEC-08). Four design rules from the SRS affect the schema directly:

- **User scoping:** Every table that holds private data has a user_id, directly or through its parent, and the API filters every query by the authenticated user (NFR-SEC-02).
- **No images:** There is no column for a product or report image. Only extracted and confirmed data is stored (NFR-SEC-05).
- **Unable to verify:** The factor columns and phss in analysis_results allow NULL. A CHECK constraint accepts NULL only when result_state is UNABLE_TO_VERIFY, so missing data cannot be saved as a zero score.
- **Deletion:** Foreign keys from a user's records use ON DELETE CASCADE, so deleting a scan also deletes its ingredients and result, and deleting a report deletes its lipid values. Deletion is a hard delete from the active store (NFR-SEC-06).

7. Entity Relationship Diagram (ERD)

Figures 9 to 11 are the logical ERD of SafeBite. They show the entities, their key attributes and the cardinality of each relationship, and they use the same entity names as the domain model and class diagrams. They are the logical view of the physical design in Section 6; attributes shown here are only the ones needed to read the relationships. The system table data_source_configs is left out because it is not related to any entity.

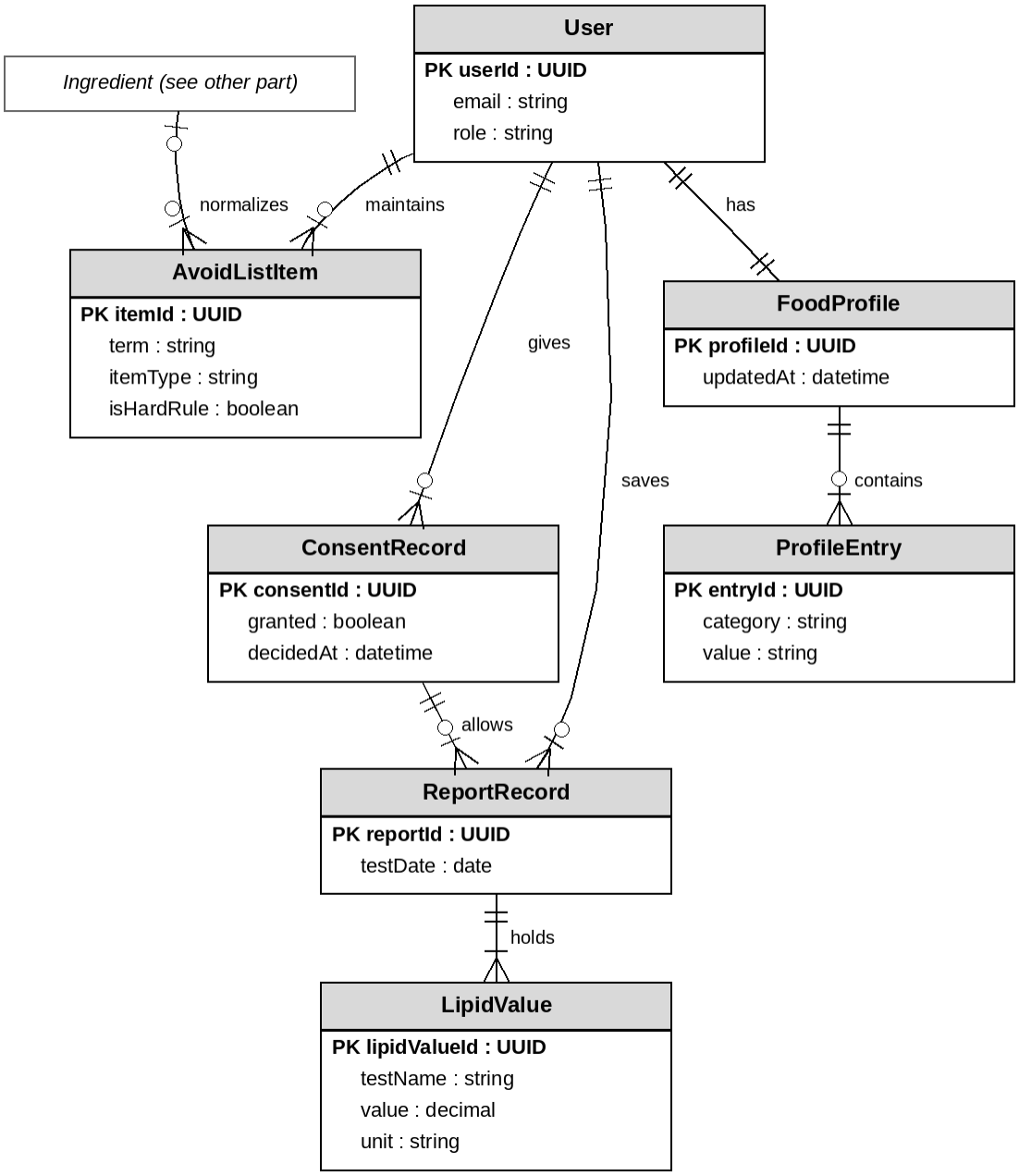


Figure 9: SafeBite entity relationship diagram: users, profile and reports.

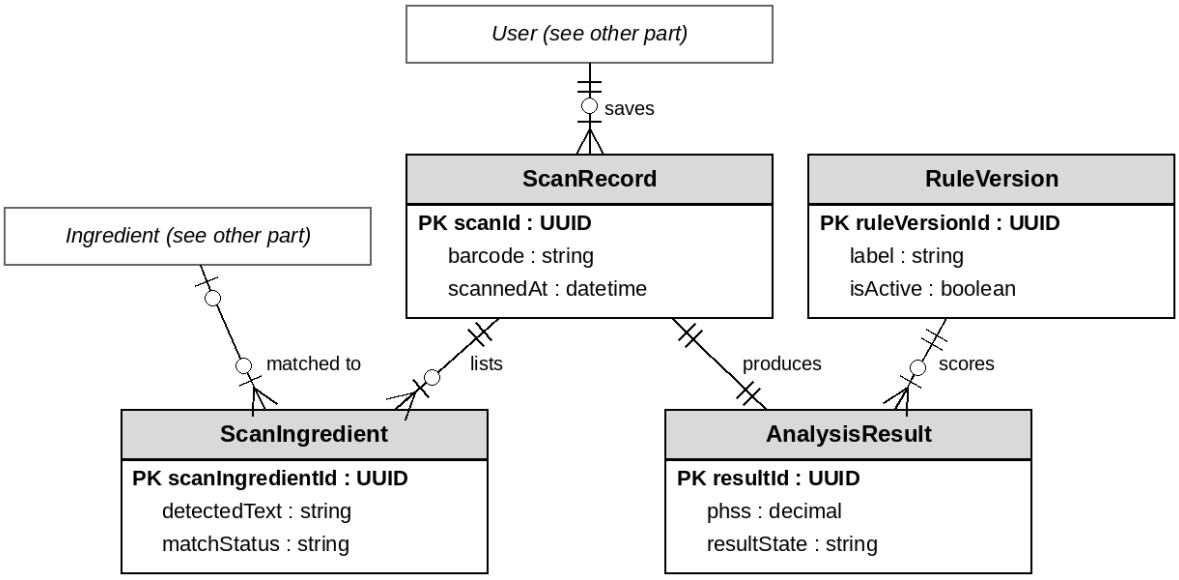


Figure 10: SafeBite entity relationship diagram: scans and results.

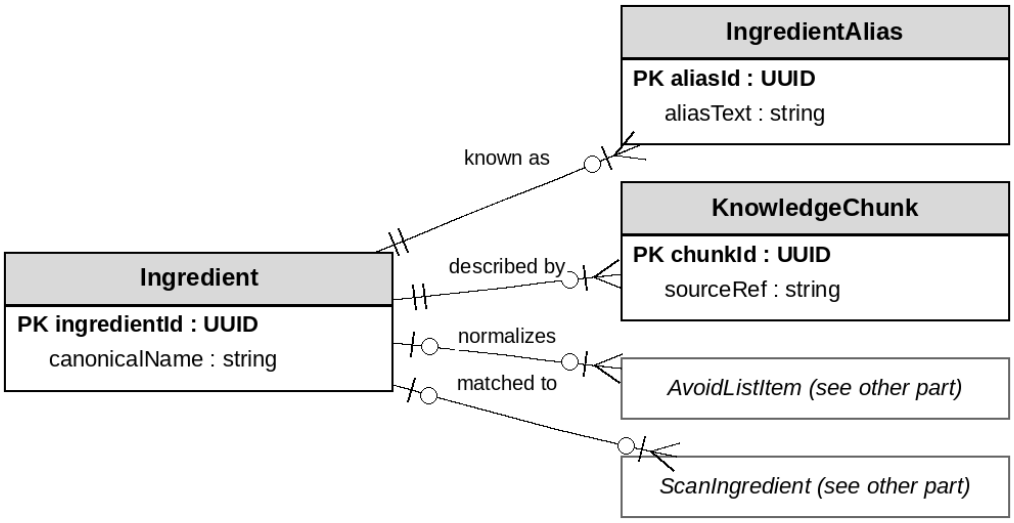


Figure 11: SafeBite entity relationship diagram: ingredient knowledge base.

Description: The ERDs show 14 entities and 15 relationships. One-to-one relationships exist between User and FoodProfile and between ScanRecord and AnalysisResult. Most other relationships are one-to-many. The optional side of Ingredient and AvoidListItem (or ScanIngredient) is shown with a circle because a user-defined term or an unmatched ingredient has no canonical record. A ReportRecord always holds at least one LipidValue and always points to the ConsentRecord that allowed it.

8. Sequence Diagram(s)

This section shows how the components in Section 3 interact during the main workflows. There is one sequence diagram for each use case in the SRS, plus one for sign-in (FR-01) and one for knowledge maintenance (SRS module 9). UC-01 and UC-06 share a diagram because both edit the profile. In every diagram, the API layer checks the ID token with AuthMiddleware before calling a service, so that step is shown once in the sign-in diagram and then written as a note. Participant names are the same as in Table 2. Return messages from a service to the app pass through the API layer and are drawn directly to keep the diagrams readable. Long diagrams continue over several pages; the participants are repeated at the top of each page.

Sign-in and Session Start

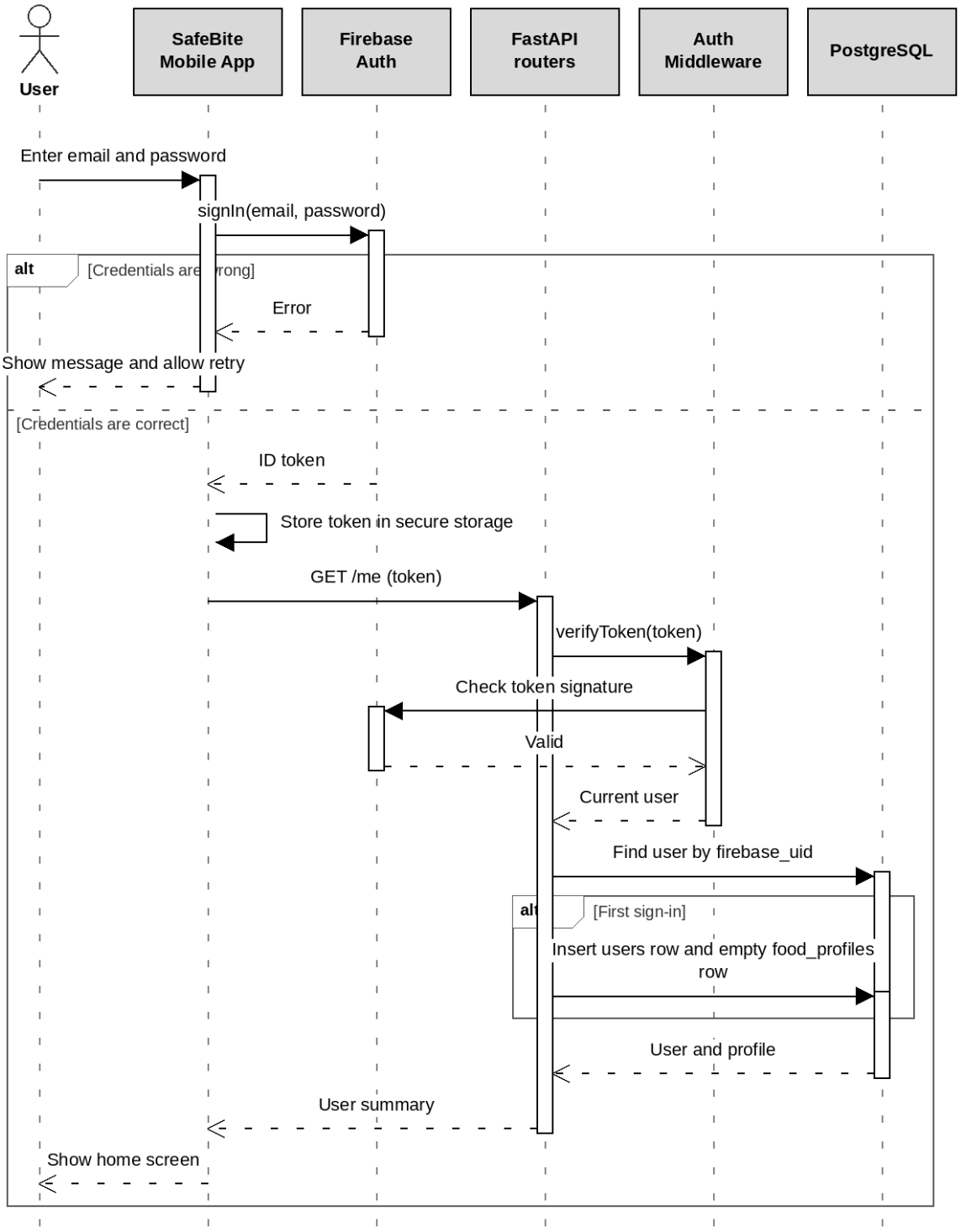


Figure 12: Sequence diagram for sign-in and session start.

Description: The user signs in with Firebase Authentication, so the SafeBite backend never sees the password. The app stores the returned ID token in secure storage and sends it to the API. AuthMiddleware verifies the token signature. On the first sign-in the API creates the users row and an empty food profile.

UC-01 / UC-06 : Create or Update Food Profile and Manage Personal Avoid List

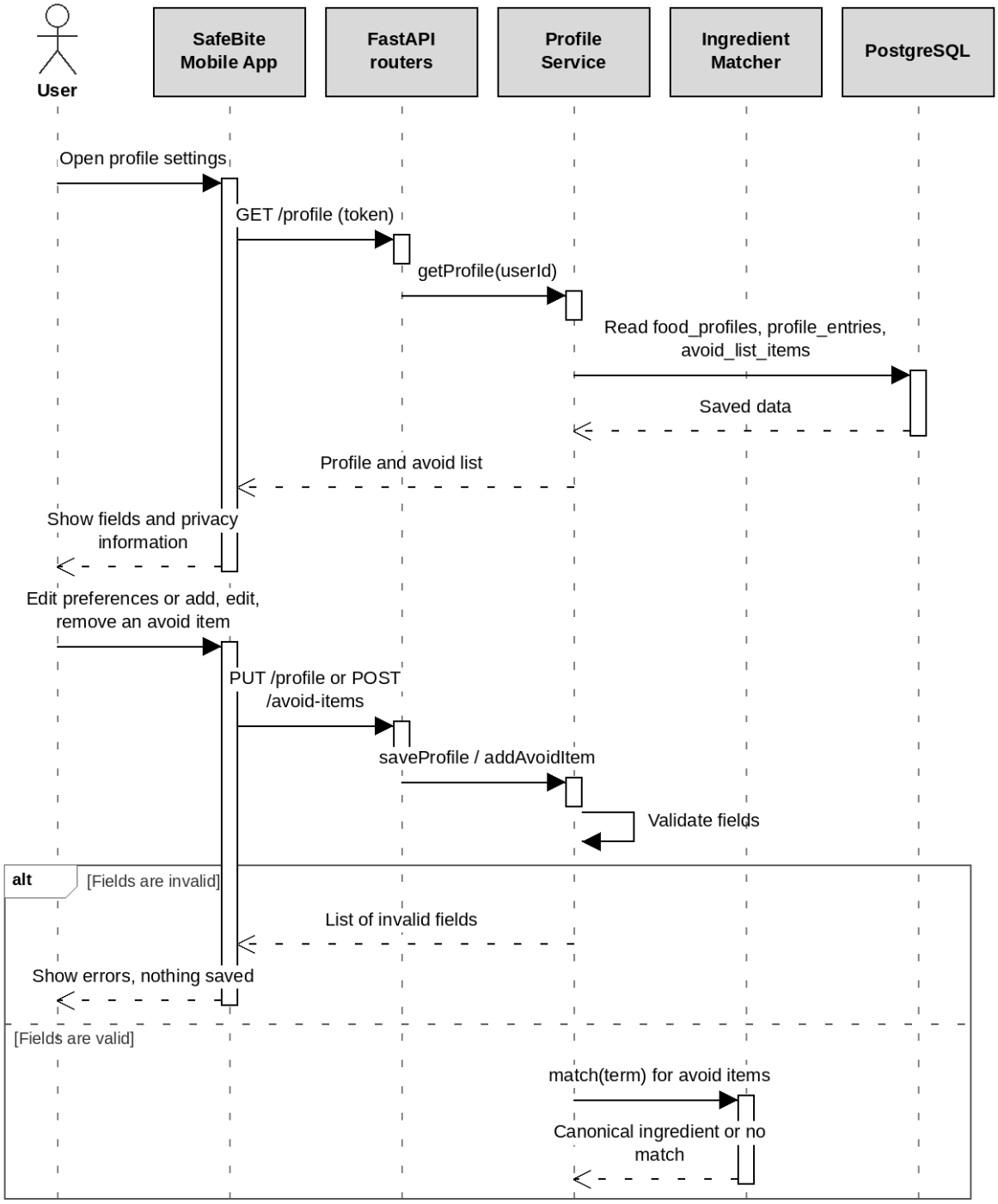


Figure 13: Sequence diagram for managing the food profile and personal avoid list. (part 1 of 2)

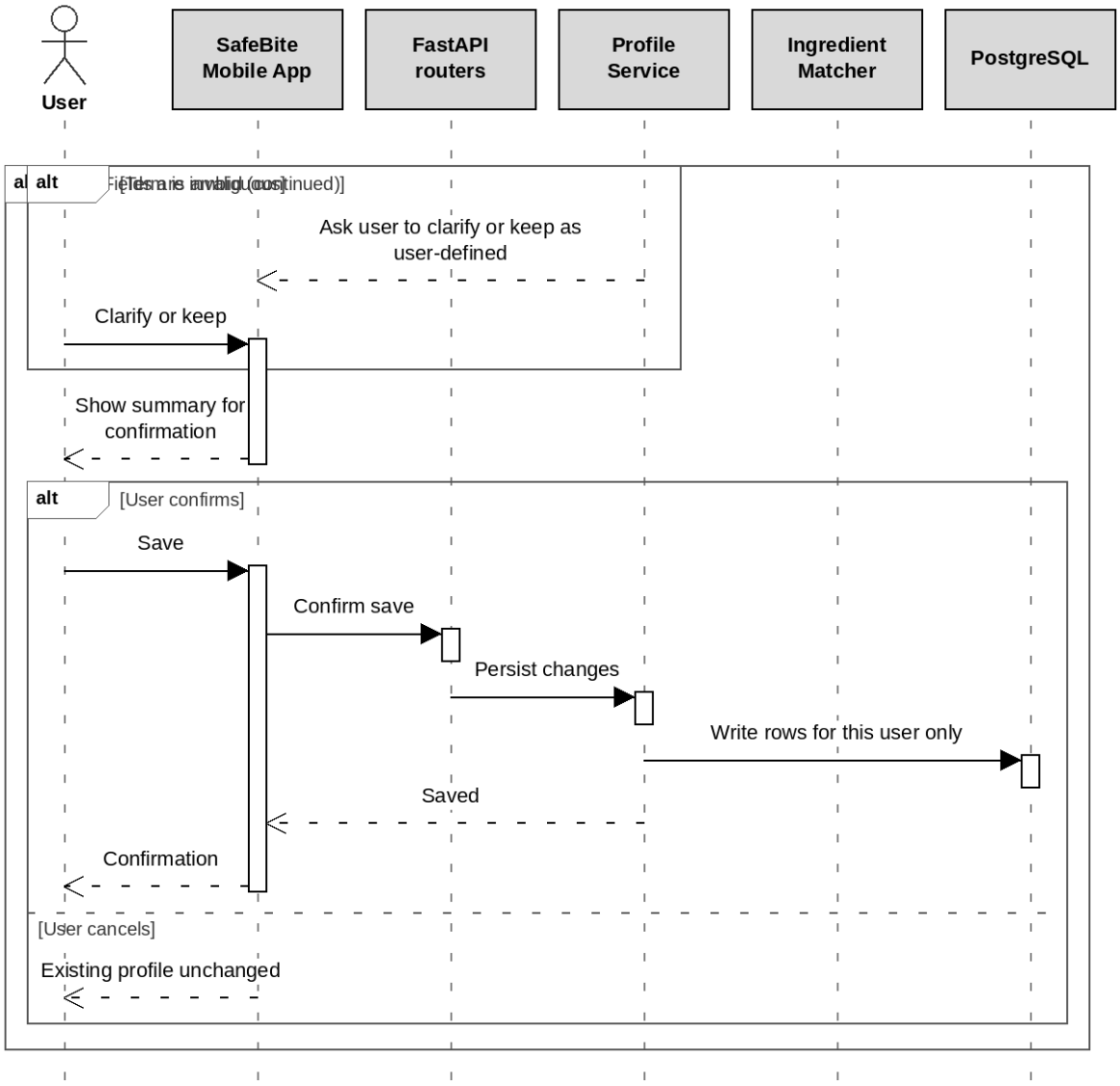


Figure 13: Sequence diagram for managing the food profile and personal avoid list. (part 2 of 2)

Description: The profile and the avoid list are saved only after the user confirms. Each avoid-list term is matched to a canonical ingredient by IngredientMatcher. A term that cannot be matched is saved as a user-defined term, and the app does not claim it is a verified allergen match. If the fields are invalid, nothing is saved; if the user cancels, the existing profile stays unchanged.

UC-02: Analyze a Packaged Food Product

The analysis is shown in three parts because of its size. The first part covers capture, product lookup, OCR and reconciliation. The second part covers loading the profile and rule version, ingredient matching, the allergen check, nutrition and evidence retrieval. The third part covers scoring, result assembly and saving.

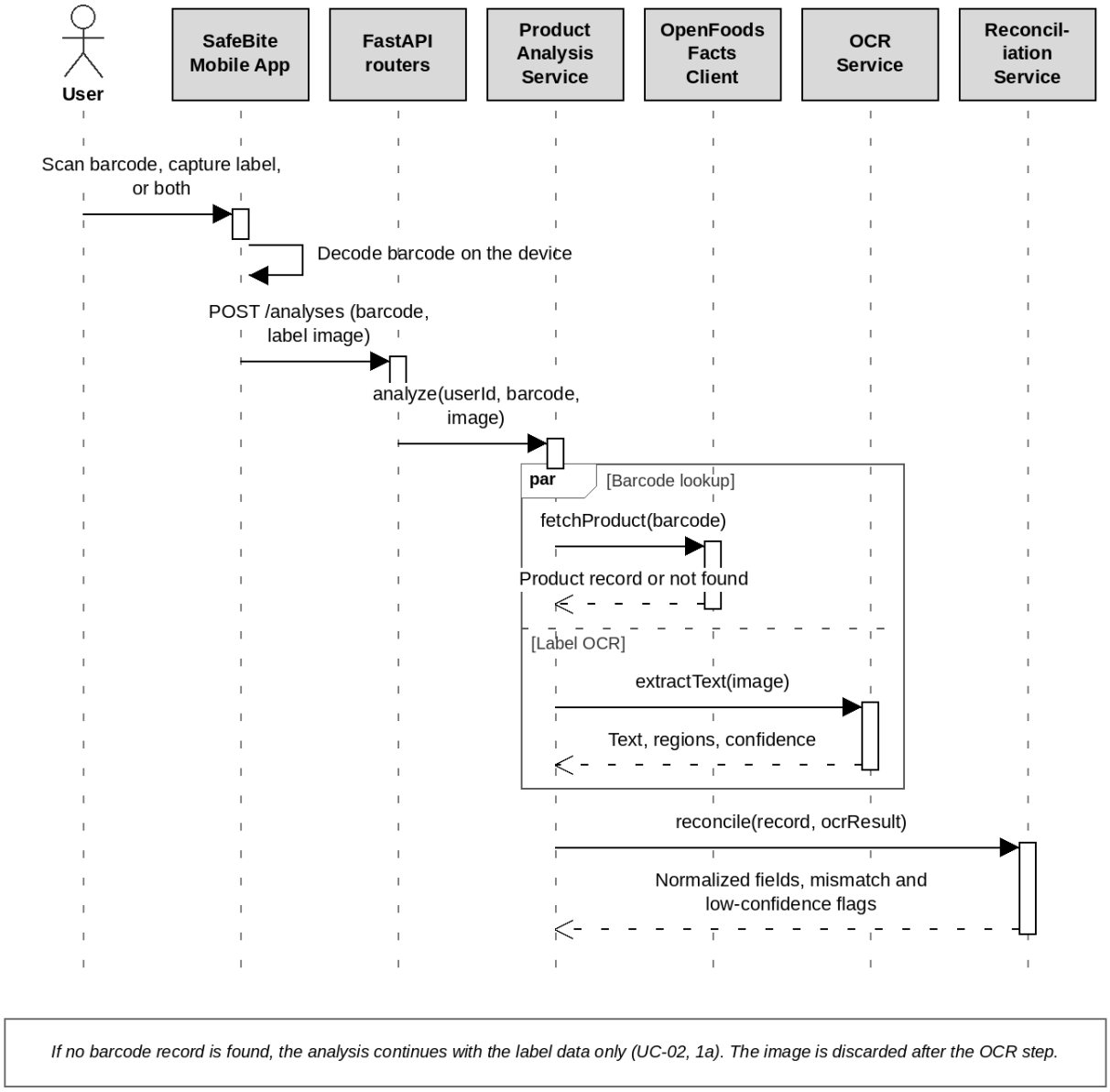


Figure 14: Sequence diagram for analyzing a packaged food product: capture and reconciliation.

Description: The barcode lookup and the OCR step run in parallel so that the result is ready within the 10 second target for most test cases. If the barcode is not found, the analysis continues with the label data only. ReconciliationService compares the database record and the OCR text and flags mismatches and low confidence. The image is not kept after the OCR step.

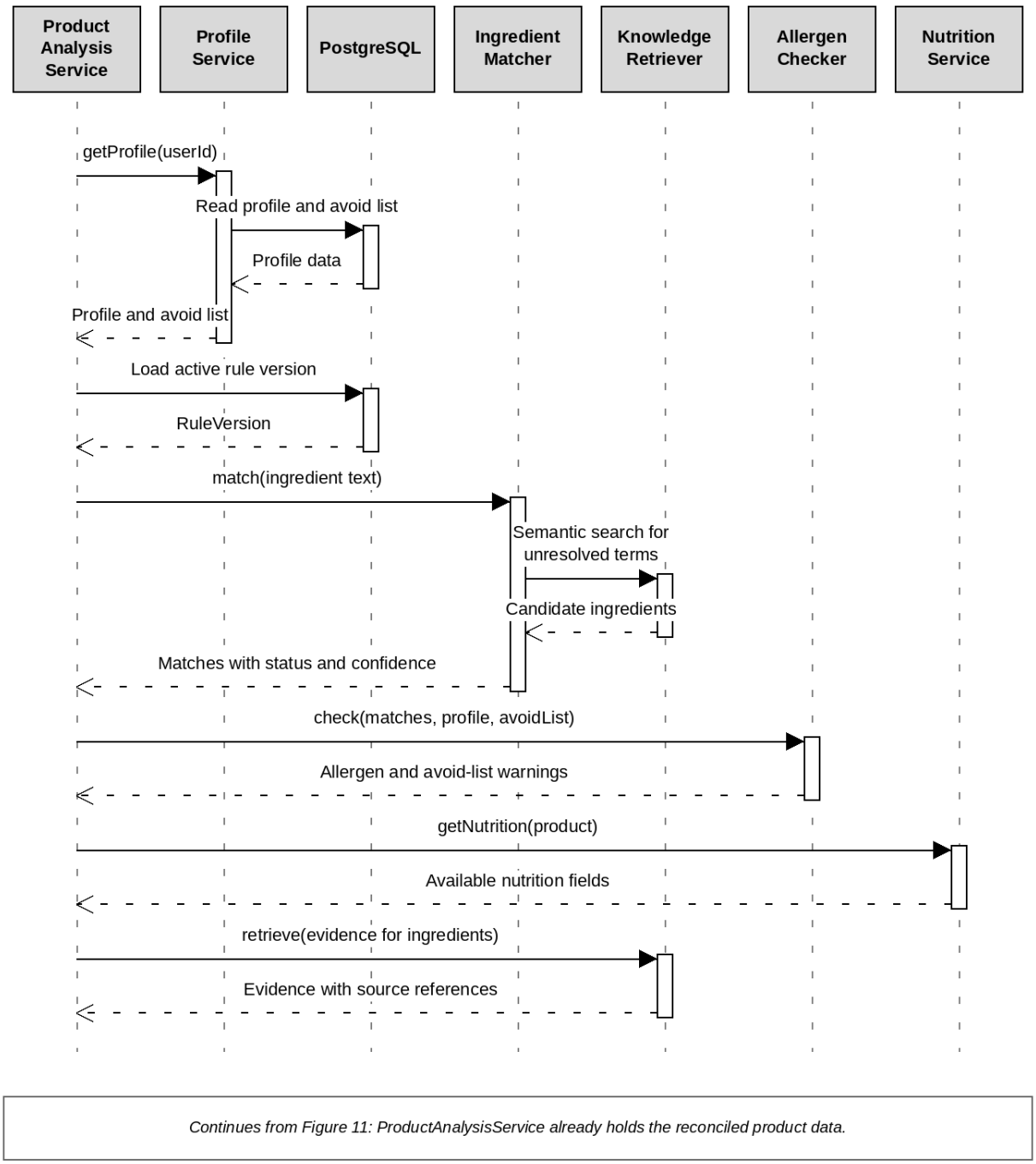


Figure 15: Sequence diagram for analyzing a packaged food product: profile, matching and checks.

Description: ProfileService supplies the profile and avoid list, and the active rule version is loaded so that the same input always gives the same result. IngredientMatcher maps ingredient text to canonical ingredients and uses KnowledgeRetriever only for terms it could not resolve. The allergen and avoid-list check runs before the score is used.

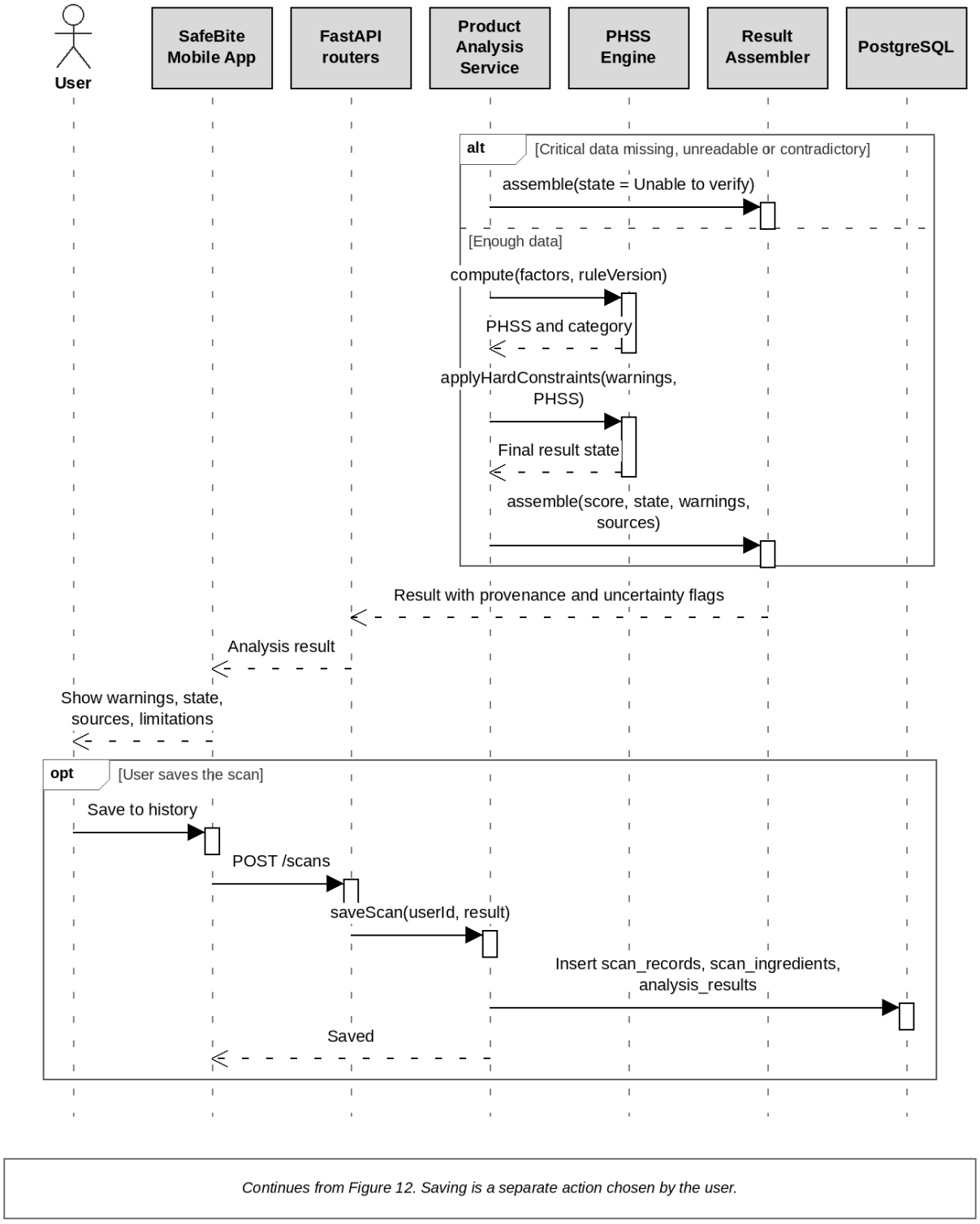


Figure 16: Sequence diagram for analyzing a packaged food product: scoring, result and saving.

Description: When the data is not enough, the result is Unable to verify instead of a score. Otherwise PHSSEngine computes the PHSS and then applies the hard constraints to decide the final state. ResultAssembler builds the result with its sources and uncertainty flags. Saving the scan is a separate action chosen by the user.

UC-03: Explore an Ingredient

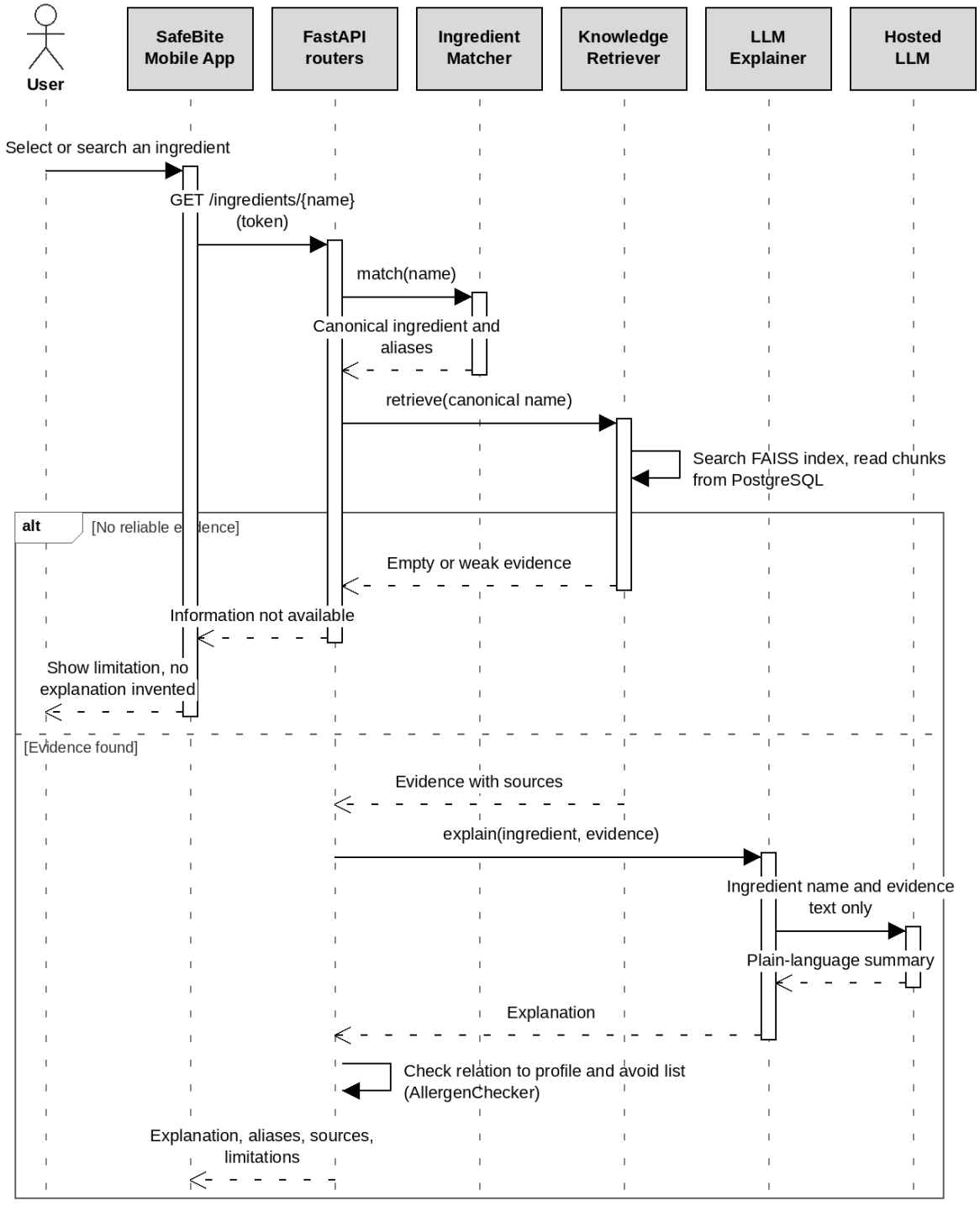


Figure 17: Sequence diagram for exploring an ingredient. (part 1 of 2)

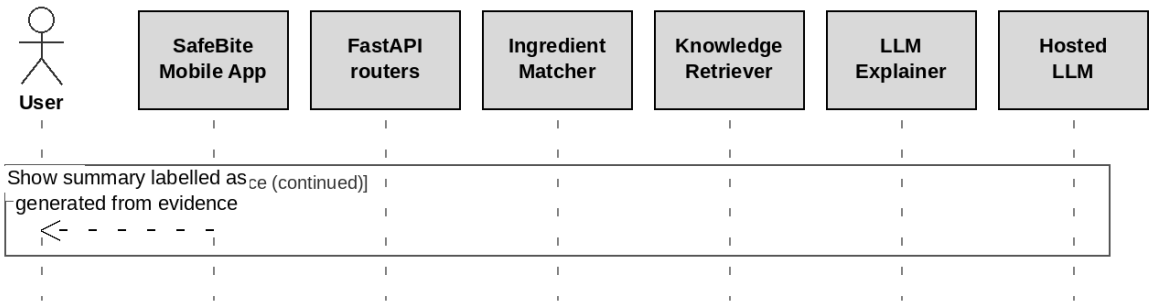


Figure 17: Sequence diagram for exploring an ingredient. (part 2 of 2)

Description: The Ingredient Explorer uses the same retrieval path as the product analysis. The LLM is called only when evidence was found, and the call contains the ingredient name and the retrieved text only. If nothing reliable is retrieved, the app says that enough information is not available and does not call the LLM. The explanation is labelled as generated from retrieved evidence.

UC-04: Compare Products

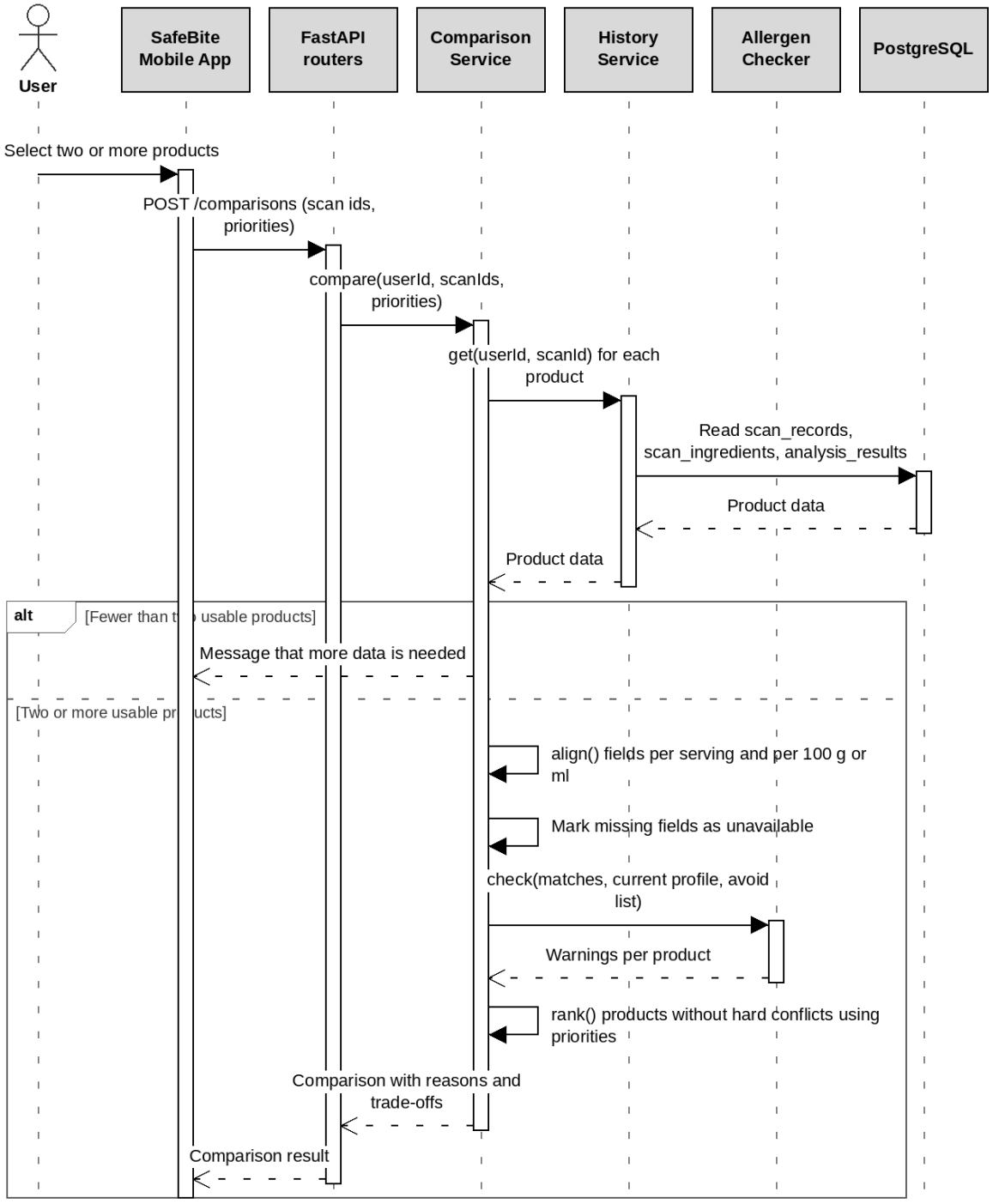


Figure 18: Sequence diagram for comparing products. (part 1 of 2)

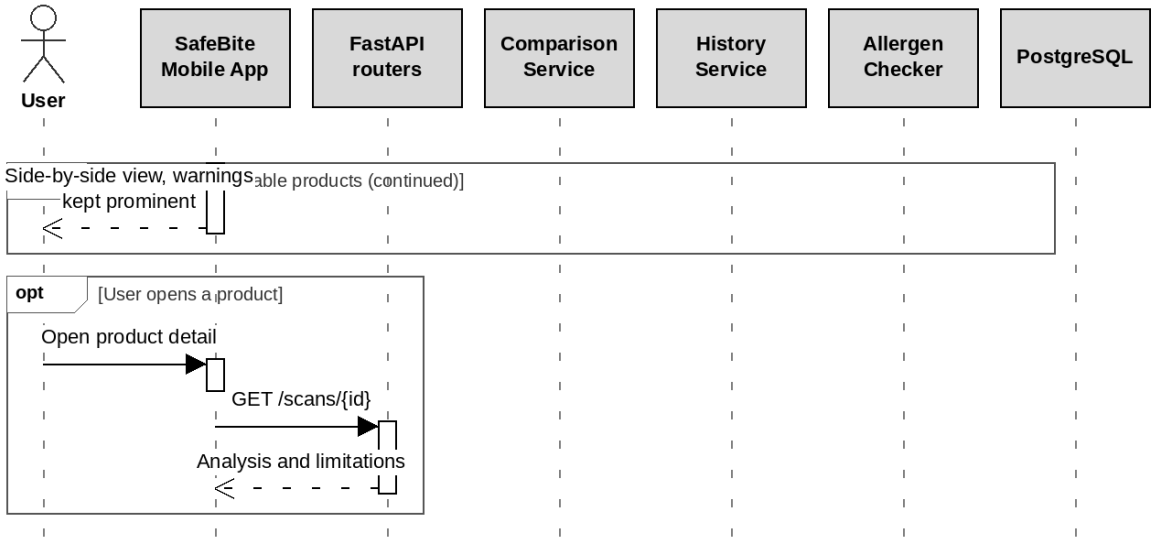


Figure 18: Sequence diagram for comparing products. (part 2 of 2)

Description: Hard constraints are applied before ranking, so a product that conflicts with a declared allergy is never promoted because of a good nutrition score. Missing fields are shown as unavailable rather than assumed equal. If fewer than two usable products exist, the user is told that more data is needed.

UC-05: Scan and Review a Health Report

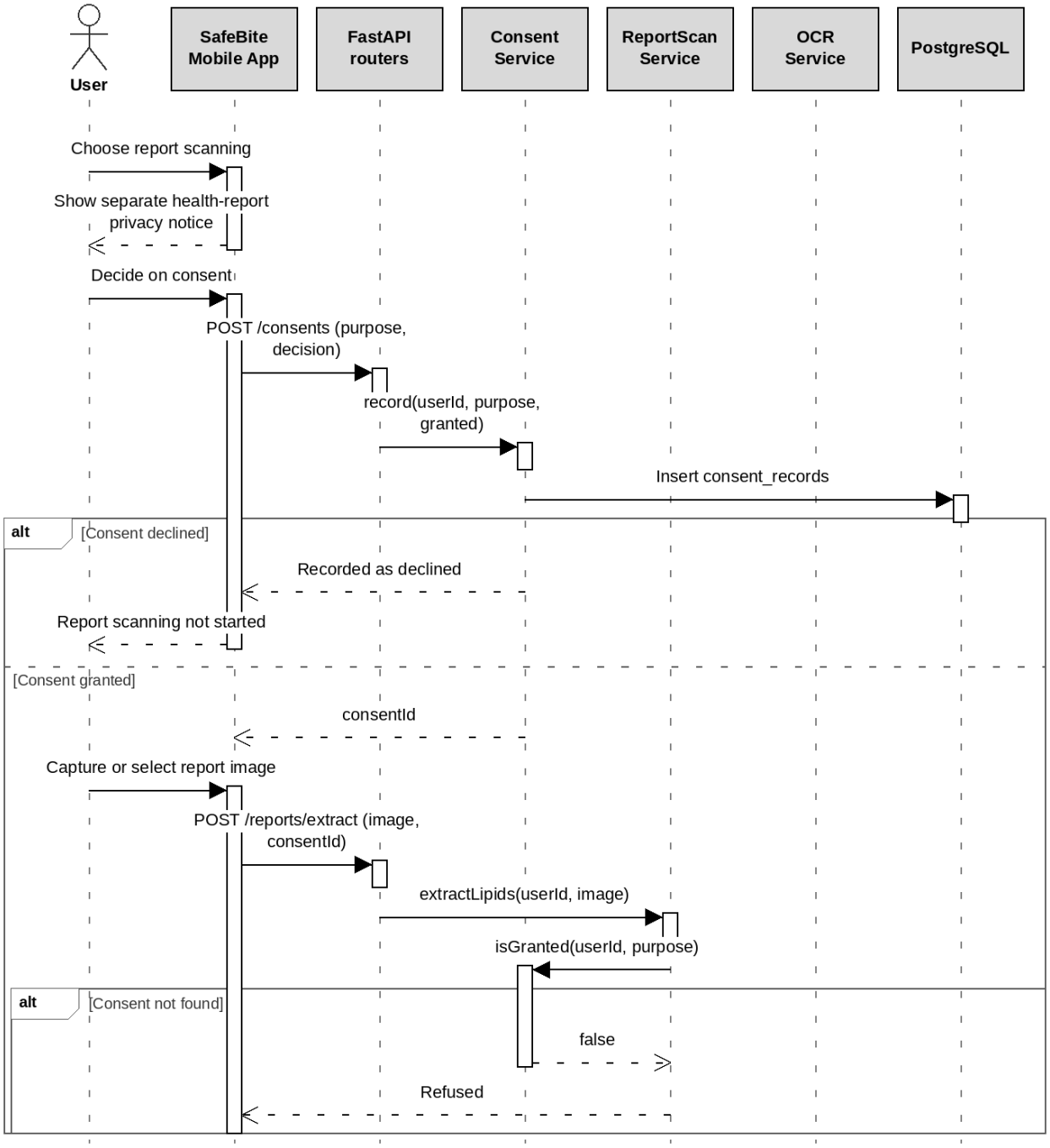


Figure 19: Sequence diagram for scanning and reviewing a health report. (part 1 of 2)

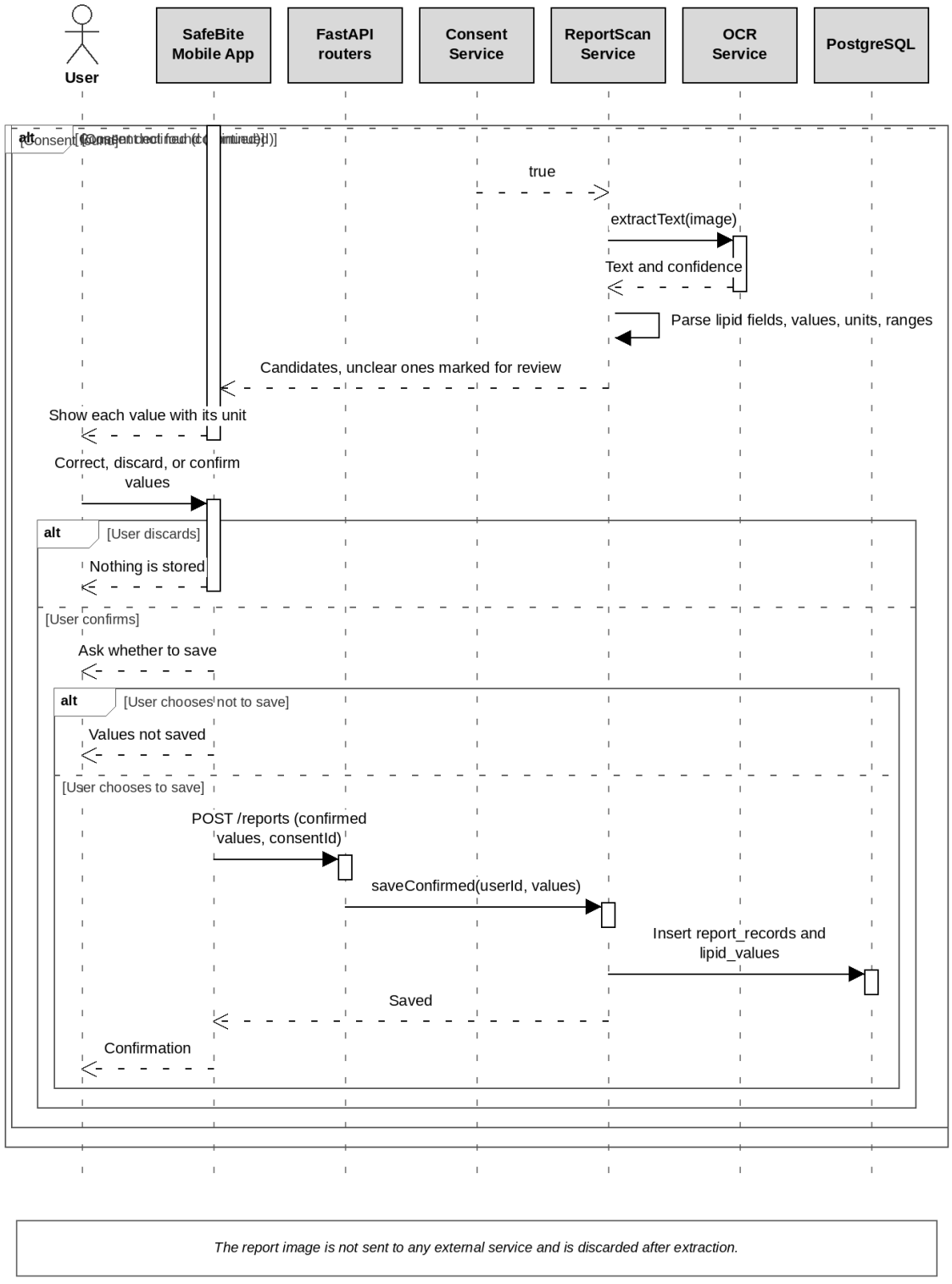


Figure 19: Sequence diagram for scanning and reviewing a health report. (part 2 of 2)

Description: Report scanning starts only after the user gives consent in a separate step. ReportScanService checks the consent again on the server before it processes the image. The extracted values are candidates

only: the user can correct or discard them, and they are saved only when the user confirms and chooses to save. The report image is not sent to any external service and is discarded after extraction.

UC-07: Review or Delete Scan History

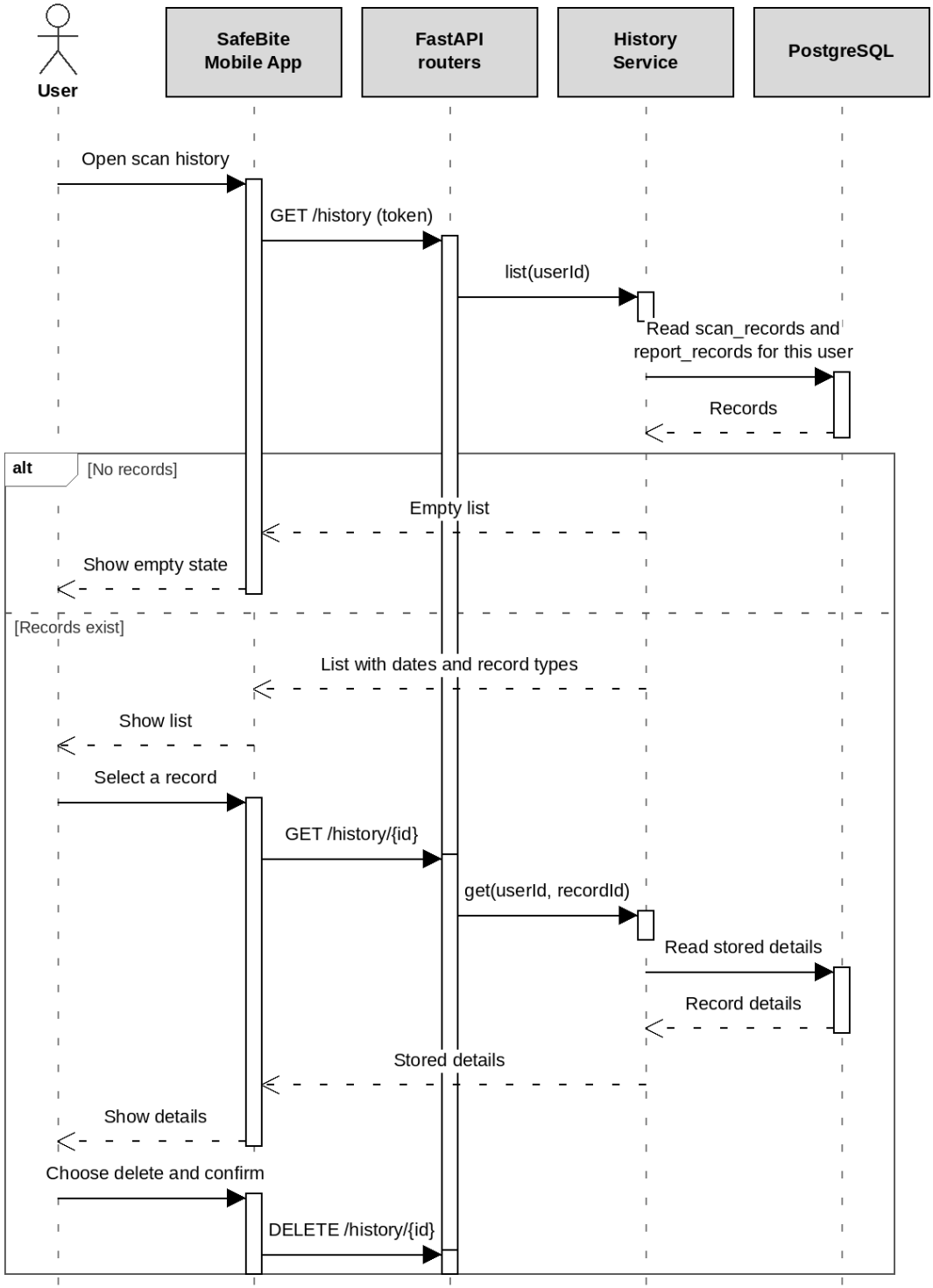


Figure 20: Sequence diagram for reviewing and deleting scan history. (part 1 of 3)

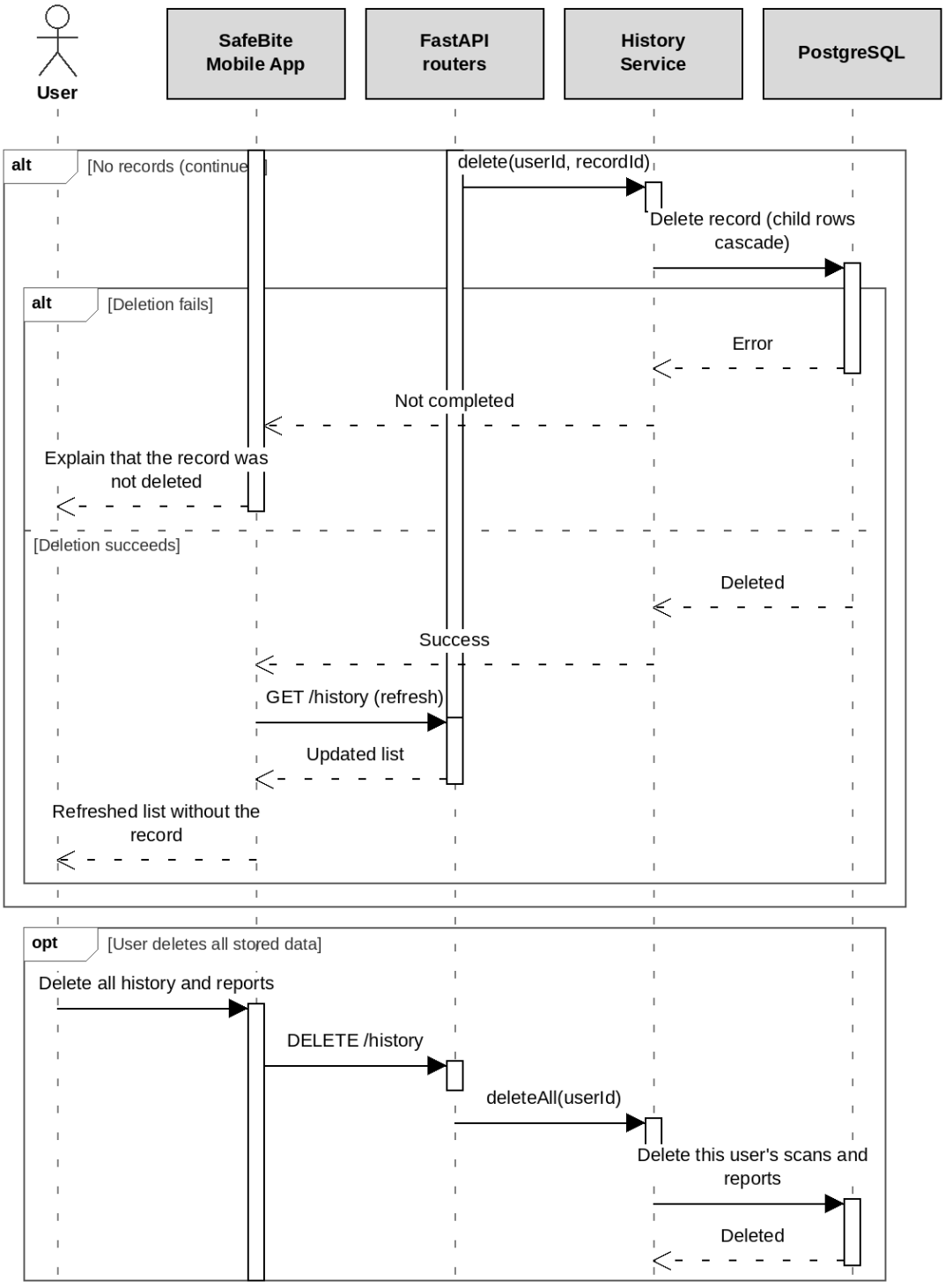


Figure 20: Sequence diagram for reviewing and deleting scan history. (part 2 of 3)

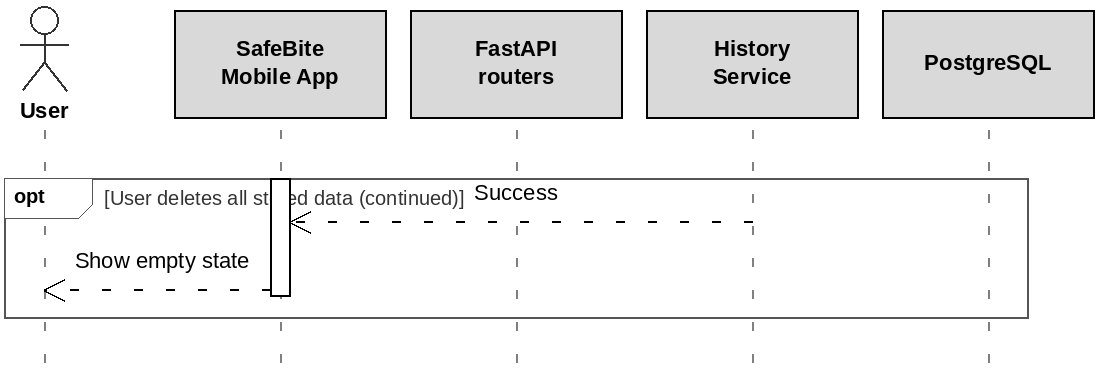


Figure 20: Sequence diagram for reviewing and deleting scan history. (part 3 of 3)

Description: The app confirms a deletion only after the database reports that the record was deleted. If the deletion fails, the app says so and does not show success. Deleting a record removes its child rows by cascade. The user can also delete all stored history and reports.

Knowledge and Rule Maintenance (SRS Module 9)

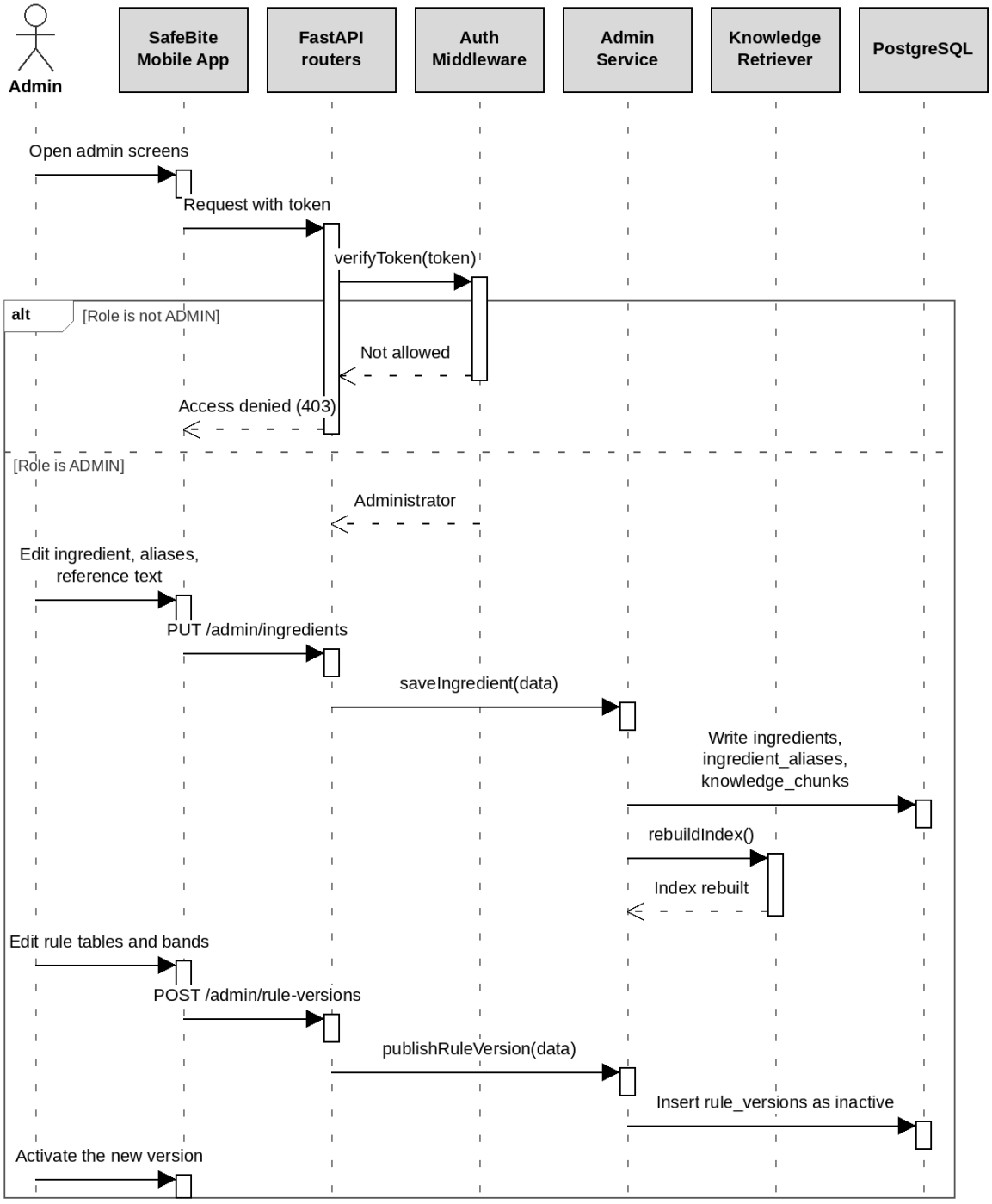


Figure 21: Sequence diagram for knowledge and rule maintenance. (part 1 of 2)

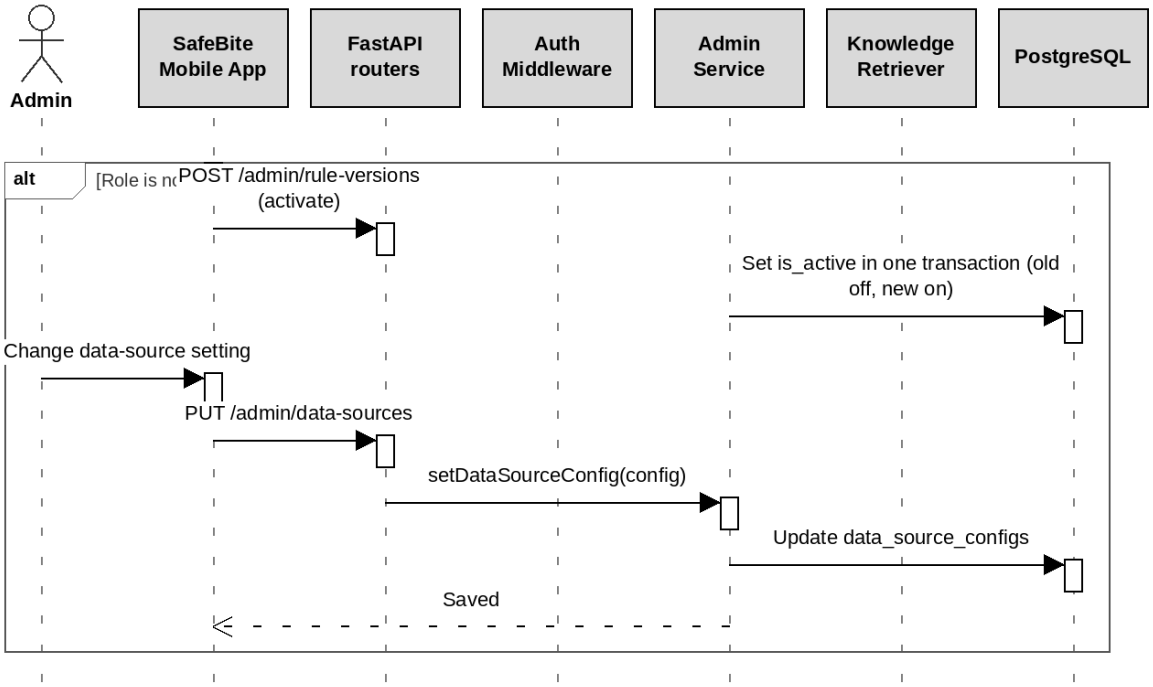


Figure 21: Sequence diagram for knowledge and rule maintenance. (part 2 of 2)

Description: The SRS lists administration and knowledge maintenance as a module but gives it no use case, so this diagram is a design assumption. AuthMiddleware refuses a non-administrator. An administrator edits ingredient records, rule versions and data-source settings. The FAISS index is rebuilt after the knowledge base changes. A new rule version is saved as inactive first, and activating it switches the old one off in the same transaction.

9. System Interface Design

The following screens are planned layouts and not images of completed software. Names and numbers in them are illustrative samples. The citizen application uses large controls and short workflows for outdoor use, with a bottom bar providing access to the five main areas.

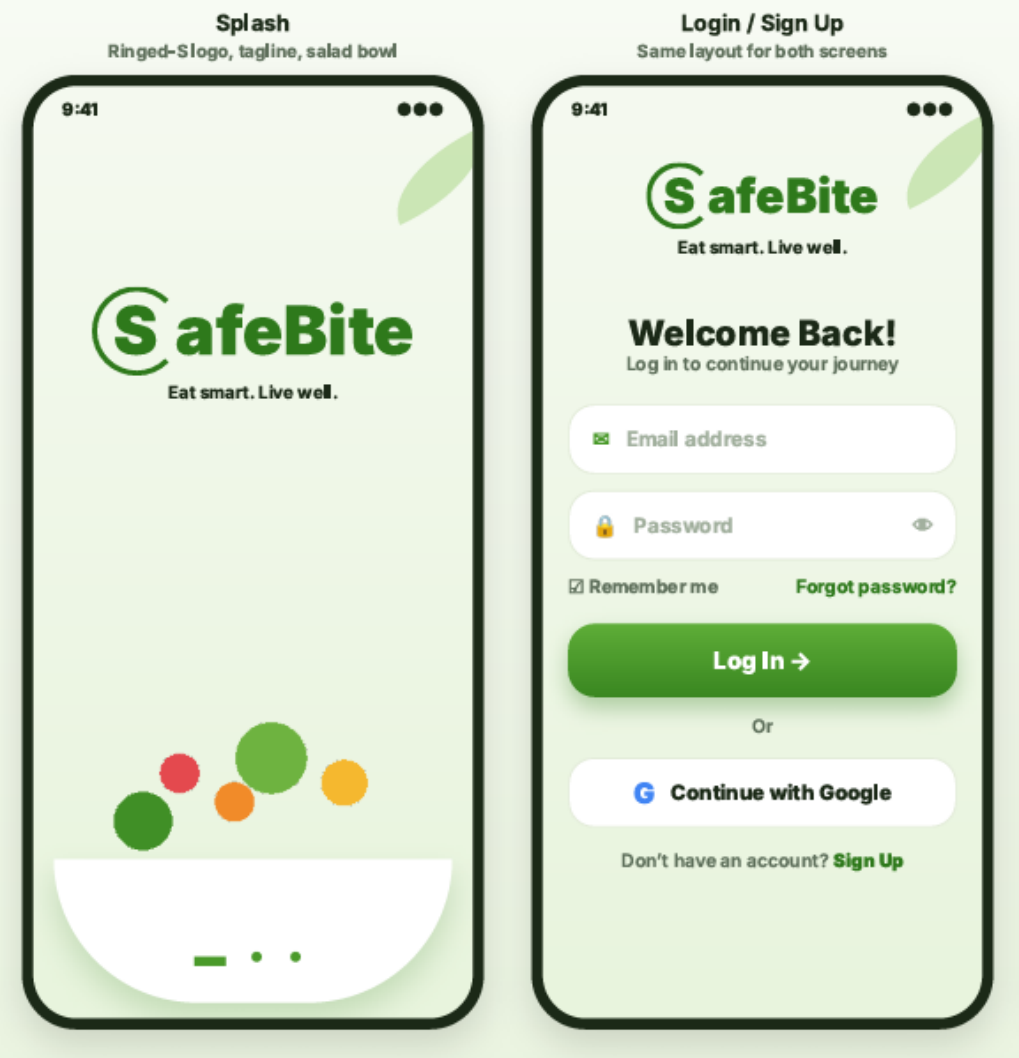


Figure 22: User Sign-in UI

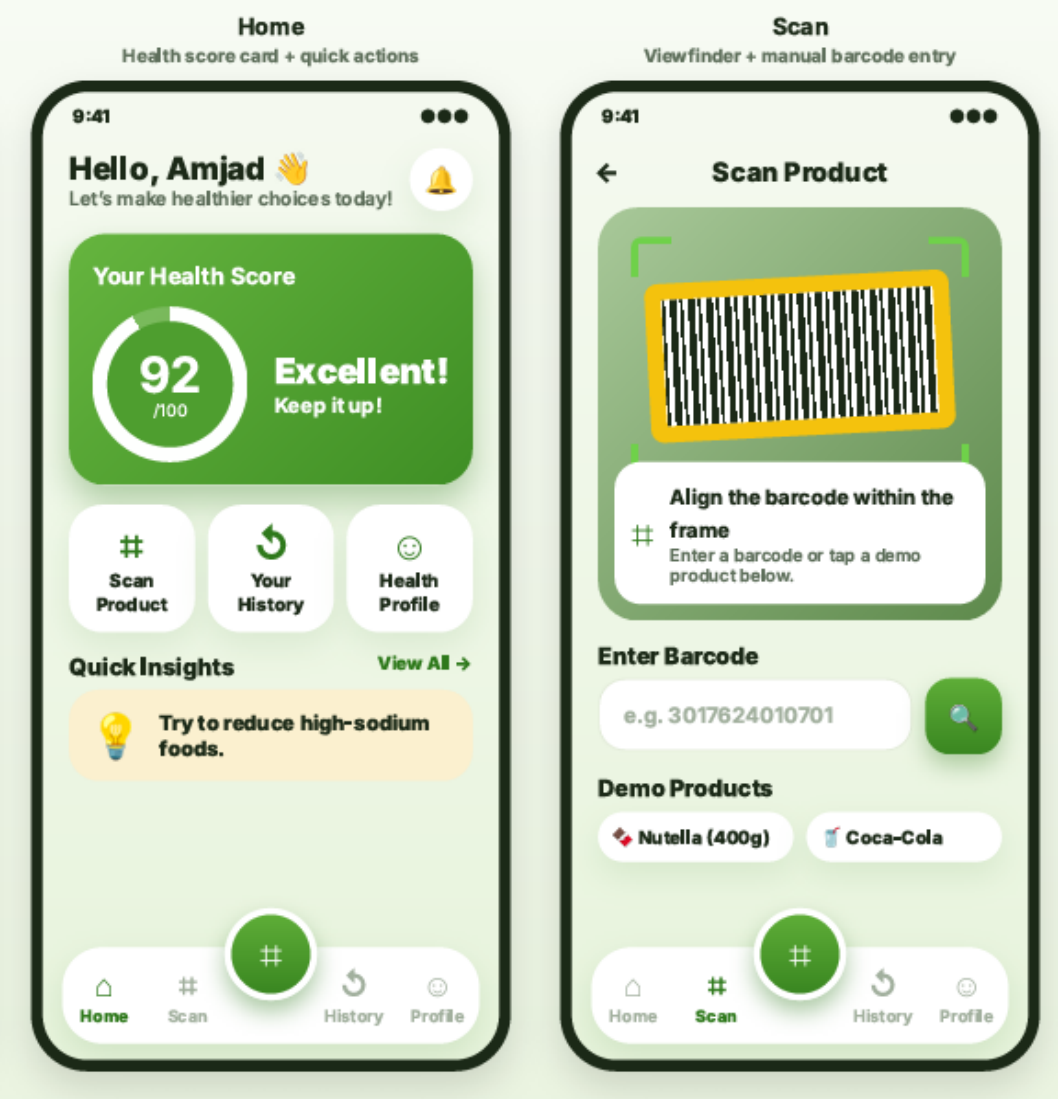


Figure 23: Home and Scan Dashboards

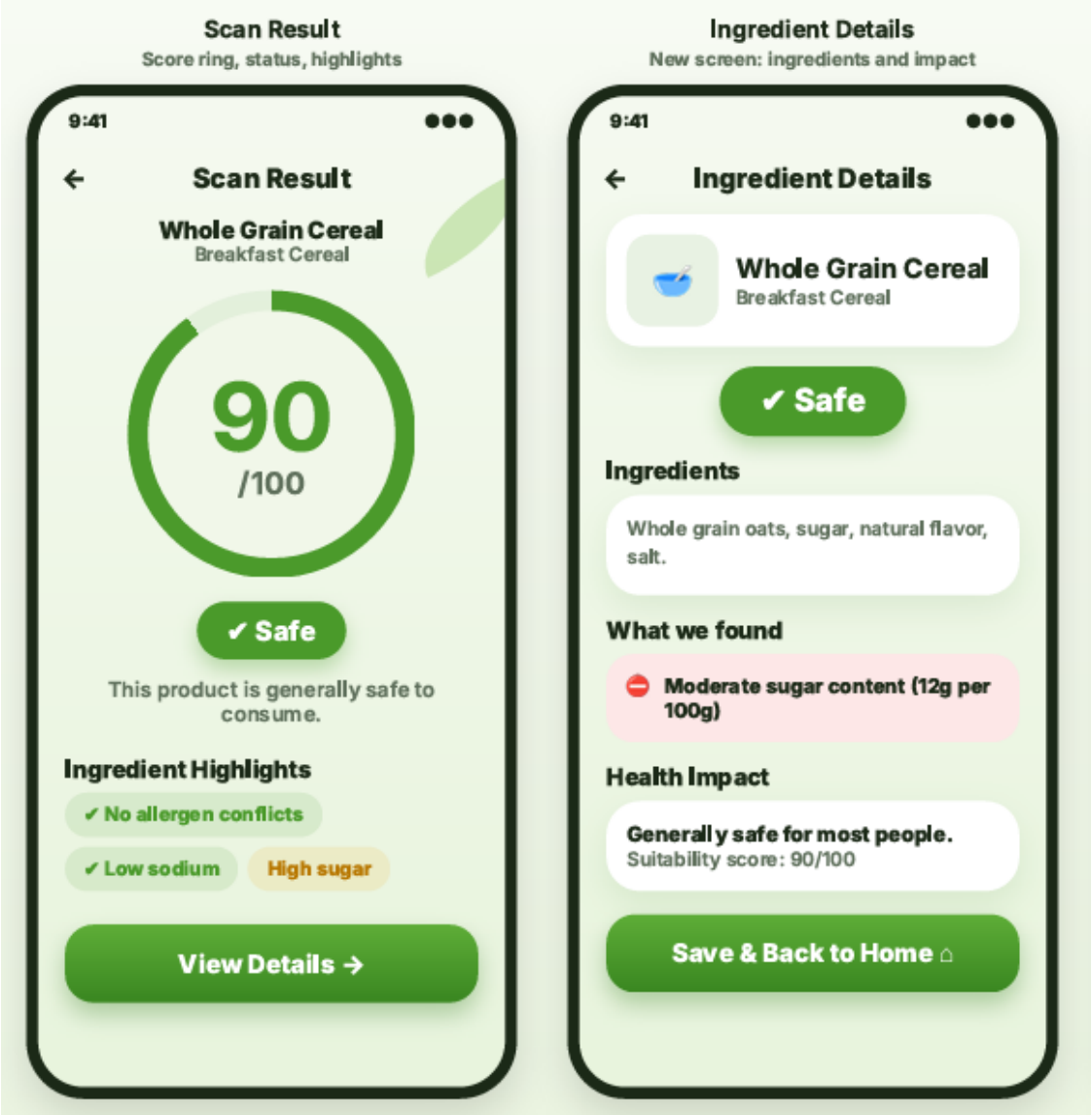


Figure 24: Scan results and Ingredient Details Dashboards



Figure 25: History and Health profile UI

Description: Figures 22 to 25 show the planned screens of the SafeBite mobile application. They follow the safety rules of the SRS: warnings and the result state are shown at the top of the result, Safe* always carries the note that no conflict was found in the available information, and every result states that SafeBite is not a medical diagnosis.

10. Test Cases

This section describes the functional and non-functional test cases designed to verify the key features, performance, security, and reliability of the SafeBite platform. These test cases ensure that the system performs as expected under different operational conditions.

10.1. Functional Test Cases

Table 3: Functional Test Cases for SafeBite Platform

Test Case ID	Module/Feature	Test Objective	Input	Expected Output	Actual Output	Status (Pass/Fail)
TC-01	User Registration & Login	Verify that users can register, sign in, and access their accounts successfully.	Valid and invalid user credentials.	User account is created, valid users are redirected to the home screen, and invalid login attempts are rejected.		
TC-02	User Profile Management	Verify that users can create and update their food preferences and health profiles.	Allergy information, dietary preferences, and nutrition goals.	Profile information is saved successfully and displayed when reopened.		
TC-03	Product Scanning	Verify that users can analyze food products using barcode scanning.	Valid product barcode.	Product information is retrieved and displayed with its source information.		
TC-04	OCR-Based Product Analysis	Test that users can analyze products by capturing or uploading ingredient label images.	Clear and unclear ingredient label images.	Ingredients are extracted and displayed; unreadable information is marked for review.		

Test Case ID	Module/Feature	Test Objective	Input	Expected Output	Actual Output	Status (Pass/Fail)
TC-05	Ingredient Verification	Verify that the system identifies and matches ingredients with its ingredient database.	Ingredient names, aliases, and unknown ingredient terms.	Ingredients are matched with their canonical names, while unknown or uncertain ingredients are marked accordingly.		
TC-06	Allergen Detection	Ensure that the system detects ingredients that conflict with the user's declared allergies.	User allergy profile and a product containing a matching allergen.	A prominent allergy warning is displayed and the product is marked as Avoid.		
TC-07	Avoid List Management	Verify that users can add personal avoid-list ingredients and receive warnings when matching products are analyzed.	Avoid-list ingredients and a matching food product.	Matching ingredients are identified and appropriate warnings are displayed.		
TC-08	PHSS Calculation	Verify that the Personal Health Suitability Score (PHSS) is calculated according to the defined scoring rules.	Product nutrition values and user health profile.	PHSS score and its corresponding suitability category are displayed correctly.		
TC-09	Ingredient Explorer	Test that users can view detailed explanations and information about food ingredients.	Selection of an ingredient from the product analysis result.	Ingredient explanation, aliases, and available source references are displayed.		

Test Case ID	Module/Feature	Test Objective	Input	Expected Output	Actual Output	Status (Pass/Fail)
TC-10	Product Comparison	Verify that users can compare two food products based on their nutritional values and ingredients.	Two analyzed food products.	Products are displayed side by side with nutritional differences and relevant warnings.		
TC-11	Health Report Scanning	Verify that users can upload health reports and extract relevant lipid profile information.	Sample health report containing cholesterol and triglyceride values.	Extracted values and units are displayed for user review and confirmation.		
TC-12	Health Report Data Management	Ensure that users can confirm, save, or discard extracted health report information.	Confirmed or discarded report values.	Only user-confirmed information is saved, while discarded information is not stored.		
TC-13	Scan History Management	Verify that users can view, access, and delete their previously saved product scans.	Saved product scans and delete request.	Saved records are displayed, and selected records are deleted successfully.		
TC-14	Notifications and Warnings	Ensure that the system displays appropriate warnings for allergens, avoid-list conflicts, and uncertain product information.	Product containing allergens, avoided ingredients, or incomplete information.	Relevant warnings and uncertainty messages are displayed to the user.		

Test Case ID	Module/Feature	Test Objective	Input	Expected Output	Actual Output	Status (Pass/Fail)
TC-15	Admin Management	Verify that administrative functions are accessible only to authorized administrators.	Admin and normal-user login credentials.	Administrators can access authorized functions, while normal users are denied access.		

10.2. Non-Functional Test Cases

Table 4: Non-Functional Test Cases for SafeBite Platform

Test Case ID	Test Objective	Expected Result	Actual Output	Status (Pass/Fail)
NFTC-01	Validate system response time during product scanning and analysis.	At least 80% of basic product analysis results are displayed within 10 seconds under normal network conditions.		
NFTC-02	Test system reliability under concurrent user requests.	Multiple users can perform product analysis simultaneously without system crashes, timeouts, or data loss.		
NFTC-03	Verify security of user authentication and data communication.	User authentication is protected, API communication uses HTTPS, and unauthorized access is restricted.		
NFTC-04	Validate user data privacy and protection.	Personal information and health reports are processed with user consent and are not exposed to unauthorized users.		
NFTC-05	Verify accuracy and reliability of ingredient recognition.	Ingredient matching, barcode verification, and OCR extraction produce results consistent with verified test data.		
NFTC-06	Test reliability of PHSS calculation and allergen detection.	The scoring rules produce consistent results, and declared allergy conflicts are never classified as Safe.		
NFTC-07	Verify system usability and interface responsiveness.	Users can navigate the application easily, and the interface remains consistent across supported screen sizes.		

Test Case ID	**Test Objective**	**Expected Result**	**Actual Output**	**Status (Pass/Fail)**
NFTC-08	Validate error handling and system recovery.	The system displays appropriate error messages and provides retry options when external APIs or services become unavailable.		